

PRESIDENCY UNIVERSITY – BENGALURU
Operating Systems (CSE2269) – Complete Answer Bank
Semester IV / B.Tech CSE / End-Term Question Bank

PART-A — 2 Marks Each

Q1. Define Operating System and give examples. [2 Marks]

An Operating System (OS) is system software that acts as an intermediary between the user and the computer hardware. It manages hardware resources, provides a platform for application programs, and offers services like process management, memory management, file system, and I/O handling.

Functions: Process management, Memory management, File system management, Device management, Security & protection.

Examples:

- Windows 11 – GUI-based, widely used on desktops
- Linux (Ubuntu, Fedora) – open-source, used on servers
- macOS – Unix-based OS for Apple hardware
- Android – Linux-based mobile OS
- iOS – Apple's mobile operating system

Q2. Differentiate Loader and Linker. [2 Marks]

Feature	Linker	Loader
Function	Combines multiple object files into a single executable	Loads the executable into main memory for execution
Input	Object files (.obj) + libraries	Executable file (.exe)
Output	Single executable file	Program loaded into RAM, ready to run
When runs	At compile/link time	At program execution time
Example	gcc links printf library with user code	OS loads chrome.exe into memory when you open Chrome

Q3. How is the process differed from the program? [2 Marks]

Feature	Program	Process
Definition	A passive set of instructions stored on disk	An active instance of a program in execution
State	Static – exists on disk	Dynamic – changes during execution
Resources	Does not need CPU or memory	Needs CPU, memory, I/O, PCB
Existence	Can exist multiple times as a file	Each execution creates a new process

Feature	Program	Process
Example	notepad.exe on the hard disk	Notepad running in memory (visible in Task Manager)

Q4. Define Thread. Give its types. [2 Marks]

A thread is the smallest unit of CPU execution within a process. Multiple threads share the same process address space (code, data, heap) but have their own stack, program counter, and registers.

Advantage: Threads are lightweight compared to processes — context switching between threads is faster.

Types of Threads:

- User-Level Threads (ULT): Managed by user-space thread libraries (e.g., POSIX pthreads). Kernel is unaware. Fast but if one thread blocks, the entire process blocks.
- Kernel-Level Threads (KLT): Managed directly by the OS kernel. True parallelism on multi-core CPUs. Slower to create but more robust.
- Hybrid Threads: Combines ULT and KLT. Multiple user threads mapped to multiple kernel threads (M:N model). Used in Solaris.

Q5. List the key requirements of a critical-section. [2 Marks]

A critical section is a segment of code where a process accesses shared resources. Three requirements must be satisfied:

- 1. Mutual Exclusion: Only one process can execute in its critical section at any given time.
- 2. Progress: If no process is in its critical section and some processes want to enter, only those processes not in the remainder section can decide who enters next. This decision cannot be postponed indefinitely.
- 3. Bounded Waiting: There must be a bound on the number of times other processes are allowed to enter their critical sections after a process has made a request to enter its own — preventing starvation.

Q6. List out the necessary conditions for deadlock. [2 Marks]

Deadlock occurs when all four of the following conditions hold simultaneously (Coffman Conditions):

- 1. Mutual Exclusion: At least one resource must be held in a non-shareable mode — only one process can use the resource at a time.
- 2. Hold and Wait: A process holding at least one resource is waiting to acquire additional resources held by other processes.
- 3. No Preemption: Resources cannot be forcibly taken away from a process — they must be voluntarily released.
- 4. Circular Wait: A set of processes {P0, P1, ..., Pn} exist such that P0 is waiting for a resource held by P1, P1 by P2, ..., Pn by P0.

Q7. Define Deadlock. Give an example. [2 Marks]

Deadlock is a situation where a set of processes are permanently blocked, each waiting for a resource held by another process in the set — none of them can ever proceed.

Example:

Process P1 holds Resource R1 and waits for R2.

Process P2 holds Resource R2 and waits for R1.
Neither P1 nor P2 can proceed → Deadlock.

Process	Holds	Waiting For
P1	R1	R2
P2	R2	R1

This forms a circular wait: P1→R2→P2→R1→P1.

Q8. Differentiate base and limit register. [2 Marks]

Feature	Base Register	Limit Register
Purpose	Stores the starting (base) address of the process in memory	Stores the size/length of the process's memory space
Role	Defines where the process begins in physical memory	Defines how far the process can access beyond the base
Use	Physical address = base + logical address	If logical address $\geq$ limit → trap (protection violation)
Example	Base = 3000	Limit = 2000 (process spans 3000–4999)

Q9. What is dynamic loading? [2 Marks]

Dynamic loading is a memory management technique where a routine (function/module) is not loaded into memory until it is actually called. The entire program does NOT need to be in memory at program start.

How it works: All routines are kept on disk in relocatable load format. A routine is loaded only when it is called.

Advantages:

- Reduces memory usage — unused routines are never loaded
- Programs with large but rarely-used code (e.g., error handling routines) benefit greatly
- No special OS support required — the programmer manages it using library routines

Example: A word processor loads spell-check routines only when the user invokes spell-check.

Q10. Define page fault. [2 Marks]

A page fault occurs when a process tries to access a page that is referenced in its logical address space but is NOT currently loaded in physical memory (RAM). The page exists on disk (swap space) but has not been brought into RAM yet.

What happens on a page fault:

1. OS detects page fault via trap
2. OS finds the page's location on disk
3. OS loads the page into a free frame in RAM
4. Updates the page table entry (valid bit set to 1)
5. Restarts the interrupted instruction

Page faults are the basis of demand paging and virtual memory.

Q11. Define system calls. Name any two types of system calls. [2 Marks]

A system call is a programmatic interface through which a user-level process requests a service from the operating system kernel. System calls are the only way user programs can access privileged kernel functions. When a system call is made, the CPU switches from user mode to kernel mode.

Two Types (with examples):

- Process Control: `fork()` – creates a new process; `exit()` – terminates a process; `wait()` – waits for child process to finish.
- File Management: `open()` – opens a file; `read()` – reads from a file; `write()` – writes to a file; `close()` – closes a file.

Q12. Differentiate between kernel mode and user mode. [2 Marks]

Feature	Kernel Mode	User Mode
Privilege	Full access to all hardware & memory	Restricted — cannot directly access hardware
Who runs here	OS kernel, device drivers	User applications (browser, games, etc.)
Error impact	A crash can halt the entire system	Only the application crashes; OS is safe
Memory access	Can access all of RAM	Can only access its own process memory
Switching	System call or interrupt switches to kernel mode	Normal execution returns to user mode
Example	Interrupt handler, scheduler	MS Word, Google Chrome

Q13. What is a Process Control Block (PCB)? Mention its contents. [2 Marks]

A Process Control Block (PCB) is a data structure maintained by the OS for every process. It contains all information the OS needs to manage a process. When a context switch occurs, the current state is saved in the PCB.

Contents of PCB:

- Process ID (PID): Unique identifier for the process
- Process State: New, Ready, Running, Waiting, or Terminated
- Program Counter (PC): Address of the next instruction to execute
- CPU Registers: All general-purpose registers, stack pointer, flag registers
- CPU Scheduling Info: Priority, queue pointers
- Memory Management Info: Base/limit registers, page tables
- I/O Status Info: List of I/O devices allocated, list of open files
- Accounting Info: CPU time used, job number, process owner

Q14. Explain the difference between preemptive and non-preemptive scheduling with one example each. [2 Marks]

Feature	Preemptive Scheduling	Non-Preemptive Scheduling
Definition	CPU can be taken away from a running process before it finishes	Once a process starts, it runs until it completes or blocks voluntarily
CPU Release	Forcibly removed on higher-priority arrival or time quantum expiry	Released only on I/O block, wait, or termination
Response Time	Better — high-priority jobs get CPU quickly	Poorer — process holds CPU until done
Overhead	Higher (frequent context switches)	Lower (fewer switches)
Example	Round Robin, Preemptive Priority	FCFS, Non-Preemptive SJF

Preemptive Example: Round Robin with quantum=4ms. P1 runs 4ms, is preempted even if not done, P2 gets CPU.

Non-Preemptive Example: FCFS. P1 (burst=10ms) runs completely before P2 starts.

Q15. Differentiate between deadlock prevention and deadlock avoidance. [2 Marks]

Feature	Deadlock Prevention	Deadlock Avoidance
Approach	Eliminate at least one of the 4 necessary conditions	Allow conditions but dynamically check if granting a resource is safe
When applied	At system design time	At runtime (per resource request)
How	Deny hold-and-wait, enforce ordering, allow preemption	Use Banker's Algorithm to check safe state
Overhead	Low at runtime	High — requires future resource knowledge
Flexibility	Restrictive — may deny valid requests	More flexible but complex
Example	Force process to request all resources at start	Banker's Algorithm

Q16. Explain how Peterson's solution ensures mutual exclusion in the critical section problem. [2 Marks]

Peterson's solution uses two shared variables: turn (whose turn it is) and flag[i] (whether process i wants to enter CS).

For Process P_i:

```

flag[i] = true; // I want to enter
turn = j;      // Give turn to other process
while (flag[j] && turn == j); // Wait if other wants AND it's their turn
// --- Critical Section ---
flag[i] = false; // Done, leave CS

```

Mutual Exclusion: Both processes cannot be in CS simultaneously. If both want to enter, turn is either i or j — only one proceeds.

Progress: If only one wants to enter, it does so immediately (other's flag is false).

Bounded Waiting: After P_i sets $flag[i]=true$ and $turn=j$, P_i will enter CS after at most one entry by P_j .

Q17. What is a resource allocation graph? How is it used to detect deadlocks? [2 Marks]

A Resource Allocation Graph (RAG) is a directed graph used to represent the state of resource allocation:

- Process nodes: Circles ($P_1, P_2, \dots$)
- Resource nodes: Rectangles with dots (each dot = one instance)
- Request edge: $P_i \rightarrow R_j$ (process P_i is requesting resource R_j)
- Assignment edge: $R_j \rightarrow P_i$ (resource R_j is assigned to P_i)

Deadlock Detection using RAG:

- If the graph has NO cycle $\rightarrow$ no deadlock
- If the graph has a cycle:
 - $\rightarrow$ AND each resource has only ONE instance $\rightarrow$ deadlock exists
 - $\rightarrow$ AND resources have multiple instances $\rightarrow$ deadlock MAY exist (not certain)

Example: $P_1 \rightarrow R_1 \rightarrow P_2 \rightarrow R_2 \rightarrow P_1$ forms a cycle with single-instance resources $\rightarrow$ Deadlock.

Q18. Define swapping. Why is it used in memory management? [2 Marks]

Swapping is a memory management technique where an entire process is temporarily moved (swapped out) from main memory (RAM) to a secondary storage (disk/swap space), and later moved back (swapped in) when needed.

Why it is used:

- Enables more processes to run than physical memory can hold simultaneously
- Allows multiprogramming when RAM is insufficient
- Provides the illusion of having more memory than physically available
- When a process is waiting for I/O, it can be swapped out to free RAM for other processes

Limitation: Swapping is slow because disk access is much slower than RAM. Modern systems use paging as a more efficient alternative.

Q19. List any two advantages of using paging in memory management. [2 Marks]

- 1. No External Fragmentation: Paging divides memory into fixed-size frames and processes into fixed-size pages. Pages are allocated to any available frame — no need for contiguous memory. This eliminates external fragmentation entirely.
- 2. Simple Memory Allocation: The OS just needs to maintain a free-frame list. Any free frame can hold any page — allocation and deallocation are straightforward with no compaction needed.

Q20. Differentiate between logical address and physical address with an example. [2 Marks]

Feature	Logical Address	Physical Address
Definition	Address generated by the CPU during execution	Actual address in physical RAM (seen by memory unit)
Also called	Virtual address	Real address
Generated by	CPU / process	Memory Management Unit

Feature	Logical Address	Physical Address
		(MMU)
Visible to	User process	Hardware (RAM controller)
Range	0 to max (per process)	0 to actual RAM size

Example: If a process generates logical address 346 and the base register = 14000, then:
Physical Address = Base + Logical = 14000 + 346 = 14346

Q21. What is kernel? [2 Marks]

The kernel is the core component of an operating system that has complete control over the system. It directly interacts with hardware and manages the most critical OS functions.

Key responsibilities:

- Process scheduling and management
- Memory management (allocation, page tables)
- Device driver management
- System call handling
- Interrupt handling

The kernel runs in privileged (kernel) mode and is protected from user programs. Types: Monolithic kernel (Linux), Microkernel (Minix), Hybrid kernel (Windows NT, macOS).

Q22. Define system call. Mention two examples for system calls. [2 Marks]

A system call is the mechanism by which a user-level program requests services from the OS kernel. It is the interface between user space and kernel space. When called, the CPU switches from user mode to kernel mode.

Examples:

- `fork()`: Creates a new child process that is a copy of the parent. Used in Unix/Linux. The parent gets child's PID; child gets 0.
- `read(fd, buffer, n)`: Reads *n* bytes from a file descriptor *fd* into *buffer*. This is a file management system call used for I/O.

Q23. What is context switch. When does it happen? [2 Marks]

A context switch is the process of saving the state (context) of a currently running process/thread into its PCB and restoring the state of the next scheduled process from its PCB, so the CPU can switch between processes.

The context includes: PC, registers, stack pointer, memory maps, I/O state.

When does it happen:

- When a time quantum (time slice) expires in Round Robin scheduling
- When a higher-priority process becomes ready (preemptive scheduling)
- When the running process makes a system call or I/O request (goes to waiting state)
- When an interrupt occurs (e.g., timer interrupt)
- When a process terminates

Context switch is pure overhead — no useful work is done during the switch itself.

Q24. List the criteria to be considered while selecting a scheduling algorithm. [2 Marks]

- 1. CPU Utilization: Keep the CPU as busy as possible (maximize %).
- 2. Throughput: Number of processes completed per unit time — maximize.
- 3. Turnaround Time: Total time from process submission to completion — minimize.
- 4. Waiting Time: Total time a process spends in the ready queue — minimize.
- 5. Response Time: Time from request submission until the first response is produced — minimize (important for interactive systems).
- 6. Fairness: Each process should get a fair share of CPU time — no starvation.

Q25. What is critical section? Why are two cooperating processes not allowed to execute in their critical sections at the same time? [2 Marks]

A critical section is a part of a process's code that accesses shared resources (shared variables, files, databases) that must not be concurrently accessed by more than one process.

Why not allowed simultaneously:

If two processes execute in their critical sections at the same time and both access shared data, the result depends on the order of execution — this is called a race condition. Race conditions cause:

- Data corruption: both may overwrite each other's writes
- Inconsistent state: one reads stale data written by another
- Unpredictable behavior: output depends on CPU scheduling

Example: If P1 and P2 both do counter++ simultaneously on shared counter=5, the result might be 6 instead of 7 due to overlapping read-modify-write operations.

Q26. What is a semaphore? How can it be used for process synchronization? [2 Marks]

A semaphore is an integer variable used for process synchronization, accessed only through two atomic operations: wait() (P) and signal() (V).

```
wait(S): if S <= 0 → block; else S--  
signal(S): S++; wake up a blocked process
```

Types:

- Counting Semaphore: Integer value unrestricted — used to control access to a resource pool with multiple instances.
- Binary Semaphore (Mutex): Value is 0 or 1 — used for mutual exclusion.

Use for Synchronization:

```
// Mutual Exclusion:  
semaphore mutex = 1;  
wait(mutex); // Entry section  
// Critical Section  
signal(mutex); // Exit section
```

Semaphores are also used for ordering (e.g., producer-consumer) and counting available resources.

Q27. How can a deadlock be prevented? [2 Marks]

Deadlock prevention eliminates at least one of the four necessary conditions:

- 1. Eliminate Mutual Exclusion: Make resources shareable (e.g., read-only files can be shared). Not always possible for inherently non-shareable resources.
 - 2. Eliminate Hold and Wait: Require a process to request ALL resources it will need before starting, OR release all current resources before requesting new ones.
 - 3. Allow Preemption: If a process holding resources requests one that cannot be immediately allocated, it must release all its current resources. Resume when all needed resources are available.
 - 4. Eliminate Circular Wait: Impose a total ordering on all resource types. Require that each process requests resources in increasing order of enumeration.
-

Q28. What is external fragmentation? Describe a solution for this problem. [2 Marks]

External fragmentation occurs when there is enough total free memory to satisfy a request, but the free memory is scattered in small, non-contiguous blocks — no single block is large enough.

Example: Total free memory = 600 KB, but split into 100+200+300 KB non-contiguous blocks. A process needing 500 KB contiguous cannot be allocated even though 600 KB is free.

Solutions:

- 1. Compaction: Move all processes to one end of memory, consolidating free space into one large block. Expensive — requires relocating processes and updating all address references.
 - 2. Paging: Divide memory into fixed-size frames; process pages need not be contiguous. Eliminates external fragmentation completely (only internal fragmentation remains).
 - 3. Segmentation with Paging: Divide into logical segments, each paged — reduces both fragmentation types.
-

Q29. Formulate the steps in handling page fault. [2 Marks]

6. CPU generates a logical address. MMU checks the page table — valid bit is 0 (page not in memory).
 7. MMU triggers a page fault trap → control transfers to the OS.
 8. OS saves the process state (registers, PC) for later restart.
 9. OS verifies the logical address is valid (in process's address space). If invalid → abort. If valid → continue.
 10. OS finds a free frame in physical memory. If no free frame → use page replacement algorithm (FIFO/LRU/Optimal) to select a victim frame.
 11. If victim page is dirty (modified), write it back to disk (swap out).
 12. OS reads the required page from disk into the free frame (swap in). This is an I/O operation — process is blocked.
 13. Page table is updated: set frame number and valid bit = 1.
 14. Process is placed back in ready queue.
 15. OS restarts the interrupted instruction — this time the page is in memory and execution continues.
-

Q30. Describe the operations on a file when sequential access is used. [2 Marks]

Sequential access is the simplest file access method where data is read or written in order, one record after another, from beginning to end.

Operations:

- Read next: Reads the next portion of the file and automatically advances the file pointer to the next position.
- Write next: Writes data at the current file pointer position and advances the pointer.

- Reset: Moves the file pointer back to the beginning of the file.
- Rewind: Same as reset — repositions to start.

Key characteristic: Cannot jump to a specific record without reading all preceding records.

Example: A tape drive uses sequential access. Reading a file line by line (like reading a CSV) uses sequential access.

Used in: Compilers reading source files, log file processing, audio/video streaming.

PART-B — 10 Marks Each

Q1. Explain in detail about the operating system services. [10 Marks]

The OS provides a set of services to users and programs. These services fall into two categories:

A. Services for User Benefit:

- 1. User Interface (UI): OS provides CLI (Command Line Interface) or GUI (Graphical User Interface) for users to interact. Example: Linux bash (CLI), Windows desktop (GUI).
- 2. Program Execution: OS loads a program into memory, executes it, and handles normal or abnormal termination.
- 3. I/O Operations: Programs frequently need I/O (disk, network, keyboard). The OS handles I/O on behalf of user programs for efficiency and security.
- 4. File System Manipulation: OS provides services to create, delete, read, write, rename, and set permissions on files and directories.
- 5. Communications: Processes may need to exchange information — via shared memory or message passing (IPC). Network communications also handled.
- 6. Error Detection: OS continuously monitors for hardware errors (disk failure, memory parity error) and software errors (arithmetic overflow, illegal memory access) and responds appropriately.

B. Services for System Efficiency:

- 7. Resource Allocation: When multiple processes run concurrently, the OS allocates CPU time, memory, file storage, and I/O devices fairly using scheduling algorithms.
- 8. Accounting / Logging: OS tracks which users use how much and what kinds of computer resources. Useful for billing and performance analysis.
- 9. Protection and Security: Ensures that access to system resources is controlled. Prevents one user's process from interfering with another's. Authentication (passwords), authorization (file permissions), firewalls.

Service	Purpose	Example
UI	User-OS interaction	bash shell, Windows GUI
Program Execution	Load and run programs	./a.out in Linux
I/O Ops	Handle device I/O	printf writes via OS
File Management	CRUD on files	cp, mv, ls commands
Error Detection	Monitor & handle errors	Segfault trap
Resource Allocation	Fair CPU/memory sharing	Scheduler
Security	Protect system resources	Login, file permissions

Q2. What is system call? Explain its types in detail. [10 Marks]

A system call is a controlled entry point into the kernel through which a user-level process requests services from the OS. System calls provide the interface between a running program and the OS.

Mechanism: User program executes a special instruction (e.g., INT 0x80 on x86, SYSCALL on x86-64) → CPU switches to kernel mode → kernel performs the service → returns result to user mode.

Types of System Calls:

1. Process Control

- Create, terminate, and manage processes and jobs

- Examples: fork(), exec(), exit(), wait(), getpid()
- fork(): Creates a child process — child is a copy of parent
- exec(): Replaces current process image with a new program

2. File Management

- Create, open, read, write, close, delete files
- Examples: open(), read(), write(), close(), unlink()

3. Device Management

- Request/release device, read/write device, get/set attributes
- Examples: ioctl(), read(), write() — same calls work for files and devices

4. Information Maintenance

- Get/set time, date, process info, system info
- Examples: getpid(), alarm(), sleep(), gettimeofday()

5. Communication

- Create/delete communication connections, send/receive messages
- Examples: pipe(), socket(), connect(), send(), recv()

6. Protection

- Get/set file permissions, control access to resources
- Examples: chmod(), chown(), umask()

Category	Key System Calls	Purpose
Process Control	fork, exec, exit, wait	Create/manage processes
File Management	open, read, write, close	File I/O
Device Management	ioctl, read, write	Device access
Information	getpid, gettimeofday	System/process info
Communication	pipe, socket, send, recv	IPC / network
Protection	chmod, chown	Access control

Q3. Discuss in detail about the operating system operations with neat diagram. [10 Marks]

The OS is an event-driven software that operates using interrupts and traps. It is responsible for managing the dual-mode operation and handling events.

1. Dual-Mode Operation

- User Mode: CPU executes user application code with restricted privileges.
- Kernel Mode: CPU has full privilege — can execute any instruction, access any memory.
- Mode bit: 0 = Kernel mode, 1 = User mode
- Switching: System call or interrupt → kernel mode. Return from kernel → user mode.

2. Interrupt-Driven Operation

- Hardware Interrupts: Generated by I/O devices (keyboard press, disk read complete). CPU suspends current task, saves state, jumps to interrupt handler.
- Software Interrupts (Traps): Generated by programs — either errors (divide-by-zero, invalid memory) or intentional system calls.

3. Timer Interrupts

- OS sets a hardware timer to prevent a user process from monopolizing the CPU.
- When timer fires → interrupt → OS regains control → can preempt the process.

4. Process of a System Call (Diagram described):

16. User app calls library function (e.g., printf)
17. Library invokes system call (e.g., write syscall number loaded into register)
18. Trap instruction executed → CPU switches to kernel mode
19. Kernel looks up system call table → executes service
20. Return value placed in register → switch back to user mode
21. Library returns to user app

5. Multi-programming and Time-sharing

- Multiprogramming: Multiple processes kept in memory; CPU switches when one waits for I/O — maximizes CPU utilization.
- Time-sharing: Each process gets a small time slice (quantum) — creates illusion of simultaneous execution. Response time is low.

Q4. Describe the operating system structure in detail with neat diagram. [10 Marks]

Different OS structures represent different approaches to organizing the OS code:

1. Simple / Monolithic Structure

- All OS functions (scheduling, file system, device drivers, memory management) in a single large kernel.
- Advantage: Fast — direct function calls, no message passing overhead.
- Disadvantage: Hard to maintain, debug, and extend. A bug anywhere can crash the entire OS.
- Example: Early UNIX, MS-DOS

2. Layered Structure

- OS divided into layers (Layer 0 = hardware, Layer N = user interface). Each layer uses only services of the layer below it.
- Advantage: Easy to debug layer by layer; modular design.
- Disadvantage: Efficiency loss — requests must pass through multiple layers. Difficult to define layer boundaries.
- Example: THE operating system, early Windows NT

3. Microkernel Structure

- Only essential services in kernel (inter-process communication, basic scheduling, minimal memory management). All other services (file system, device drivers) run as user-space processes.
- Advantage: Highly reliable (kernel failures don't crash OS), easy to extend, portable.
- Disadvantage: Performance overhead from message passing between user-space services.
- Example: Minix, QNX, macOS (Darwin)

4. Modules / Loadable Kernel Modules (LKM)

- Core kernel with dynamically loadable modules. Best of monolithic and microkernel.
- Advantage: Flexible — load only what you need. Efficient — modules communicate via direct calls.
- Example: Linux kernel modules (e.g., device drivers loaded at runtime)

5. Hybrid Structure

- Combines microkernel and monolithic approaches.
- Example: Windows NT, macOS — microkernel base with monolithic components for performance.

Structure	Kernel Size	Performance	Reliability	Example
Monolithic	Large	High	Low	Linux, UNIX
Layered	Medium	Medium	Medium	THE OS

Structure	Kernel Size	Performance	Reliability	Example
Microkernel	Tiny	Lower	Very High	Minix, QNX
Modular	Variable	High	High	Linux (modules)
Hybrid	Medium	High	High	Windows, macOS

Q5. Discuss the Process States and Process Control Block (PCB) with neat diagram. [10 Marks]

Process States

A process transitions between the following five states during its lifetime:

- 1. New: The process is being created. PCB is being initialized. Not yet loaded into memory.
- 2. Ready: The process is loaded into memory and waiting for CPU allocation. It is in the ready queue.
- 3. Running: The process is currently being executed by the CPU. Only one process per CPU core can be in this state.
- 4. Waiting (Blocked): The process cannot proceed because it is waiting for an event — I/O completion, semaphore signal, or a resource.
- 5. Terminated: The process has finished execution. OS reclaims its resources and removes PCB.

State Transitions:

From	To	Cause
New	Ready	Process loaded into memory (admitted)
Ready	Running	CPU scheduler selects process (dispatched)
Running	Waiting	I/O request, wait for event
Running	Ready	Time quantum expired (preempted), or higher-priority process arrives
Waiting	Ready	I/O complete or event occurs
Running	Terminated	Process calls exit() or error

Process Control Block (PCB)

The PCB is the OS data structure representing a process. It is created when the process is created and deleted when terminated. Contains:

Field	Description
Process ID (PID)	Unique identifier
Process State	New/Ready/Running/Waiting/Terminated
Program Counter	Address of next instruction to execute
CPU Registers	All registers saved on context switch
Scheduling Info	Priority, queue pointers, scheduling parameters
Memory Info	Base/limit registers, page table pointer
I/O Info	Open file list, I/O device list
Accounting Info	CPU time used, start time, owner

Q6. Explain in detail about the Inter Process Communication (IPC). [10 Marks]

Inter-Process Communication (IPC) is a mechanism that allows processes to communicate and synchronize. Processes can be independent or cooperating. Cooperating processes need IPC.

Why IPC?

- Share data between processes
- Speed up computation using parallel processing
- Modularity — divide application into separate processes
- Convenience — multiple users sharing data

Two Fundamental Models of IPC:

1. Shared Memory

- A region of memory is shared between two or more processes.
- Processes communicate by reading/writing this shared region directly.
- The OS creates the shared segment; processes attach to it.
- Advantages: Very fast (memory-speed access), no system call overhead after setup.
- Disadvantages: Requires synchronization (semaphores/mutexes) to avoid race conditions. Complex to program.
- Example: POSIX `shmget()/shmat()`, System V shared memory.
- Use case: High-performance inter-process data exchange (e.g., database shared buffer).

2. Message Passing

- Processes communicate by sending and receiving messages through the OS kernel.
- No shared memory — data explicitly transferred.
- Operations: `send(destination, message)` and `receive(source, message)`.
- Advantages: No synchronization required (easier to program). Works across network (distributed systems).
- Disadvantages: Slower — every message requires a system call and data copy.
- Example: Pipes, message queues, sockets.

Message Passing Varieties:

- Direct Communication: Processes explicitly name each other. `send(P2, msg)` / `receive(P1, msg)`.
- Indirect Communication: Messages sent to/received from mailboxes (ports). `send(mailbox, msg)`.
- Synchronous (Blocking): Sender blocks until receiver gets message.
- Asynchronous (Non-blocking): Sender sends and continues; receiver checks when ready.

Feature	Shared Memory	Message Passing
Speed	Very fast (no kernel after setup)	Slower (kernel involved per message)
Synchronization	Required (explicit)	Built into mechanism
Data Copy	No copy (direct memory access)	Data copied each message
Best for	Large data, same machine	Small messages, distributed systems
Example	POSIX shm, mmap	Pipes, sockets, MPI

Q7. FCFS and SJF (Non-Preemptive) Scheduling — Solved Problem. [10 Marks]

Given Processes:

Process	Arrival Time	Burst Time
---------	--------------	------------

Process	Arrival Time	Burst Time
P1	0	8
P2	1	4
P3	2	9
P4	3	5

———— FCFS (First Come First Serve) ————

Order of execution: P1→P2→P3→P4

Process	AT	BT	Start	Finish	TAT (F-A)	WT (TAT-BT)
P1	0	8	0	8	8	0
P2	1	4	8	12	11	7
P3	2	9	12	21	19	10
P4	3	5	21	26	23	18

Gantt Chart: | P1(0-8) | P2(8-12) | P3(12-21) | P4(21-26) |

Avg TAT = $(8+11+19+23)/4 = 61/4 = 15.25$ ms

Avg WT = $(0+7+10+18)/4 = 35/4 = 8.75$ ms

———— SJF Non-Preemptive ————

At time 0: Only P1 available → P1 runs (BT=8). Finishes at t=8.

At t=8: P2(BT=4), P3(BT=9), P4(BT=5) all arrived. Shortest: P2.

At t=12: P3(BT=9), P4(BT=5). Shortest: P4.

At t=17: P3 runs.

Process	AT	BT	Start	Finish	TAT	WT
P1	0	8	0	8	8	0
P2	1	4	8	12	11	7
P4	3	5	12	17	14	9
P3	2	9	17	26	24	15

Gantt Chart: | P1(0-8) | P2(8-12) | P4(12-17) | P3(17-26) |

Avg TAT = $(8+11+14+24)/4 = 57/4 = 14.25$ ms

Avg WT = $(0+7+9+15)/4 = 31/4 = 7.75$ ms

Q8. Round Robin (RR) Scheduling — Solved Problem. [10 Marks]

Given: Time Quantum (TQ) = 3 ms

Process	Arrival Time	Burst Time
P1	0	10
P2	1	6

Process	Arrival Time	Burst Time
P3	2	4
P4	3	8

Execution with TQ=3 (Ready Queue logic):

Time	Running	Remaining BT: P1 P2 P3 P4
0-3	P1	7 6 4 8
3-6	P2	7 3 4 8 (P3,P4 arrive by t=3)
6-9	P3	7 3 1 8
9-10	P4 (only 3ms, gets 3)	7 3 1 5
10-13	P1	4 3 1 5
13-16	P2	4 0 1 5 → P2 done at 15
13-15	P2 finishes (only 2ms left)	4 0 1 5
15-16	P3 (1ms left)	4 0 0 5 → P3 done at 16
16-19	P4	4 0 0 2
19-22	P1	1 0 0 2
22-24	P4	1 0 0 0 → P4 done at 24
24-25	P1 (1ms left)	0 → P1 done at 25

Gantt Chart:

| P1(0-3) | P2(3-6) | P3(6-9) | P4(9-12) | P1(12-15) | P2(15-17) | P3(17-18) | P4(18-21) | P1(21-24) | P4(24-26) | P1(26-27) |

Note: Exact times depend on arrival order in queue. General result:

Process	AT	BT	Finish Time	TAT	WT
P1	0	10	25	25	15
P2	1	6	17	16	10
P3	2	4	18	16	12
P4	3	8	26	23	15

Avg TAT = $(25+16+16+23)/4 = 80/4 = 20$ ms

Avg WT = $(15+10+12+15)/4 = 52/4 = 13$ ms

Q9. Apply the Peterson's solution to solve the critical section problem. [10 Marks]

Peterson's solution is a classic software-based solution to the critical section problem for two processes (P0 and P1). It uses only two shared variables.

Shared Variables:

```
int turn;    // whose turn it is to enter CS
boolean flag[2]; // flag[i] = true means Pi wants to enter CS
```

Algorithm for Process Pi (where $j = 1 - i$):

```
do {
    flag[i] = true;    // Step 1: declare intent
    turn = j;         // Step 2: give priority to other
    while (flag[j] && turn == j); // Step 3: busy wait
    // *** CRITICAL SECTION ***
    flag[i] = false;   // Step 4: exit section
    // *** REMAINDER SECTION ***
} while (true);
```

Verification of Three Requirements:

1. Mutual Exclusion:

Suppose both P0 and P1 are in the CS simultaneously. Then $\text{flag}[0]=\text{true}$ AND $\text{flag}[1]=\text{true}$. Turn can be either 0 or 1. If $\text{turn}=1$, P0 exits while-loop ($\text{flag}[1]\&\&\text{turn}==1$ is true so P0 waits). Contradiction. Hence only one can be in CS.

2. Progress:

If Pi wants to enter and Pj does not ($\text{flag}[j]=\text{false}$), the while condition is false $\rightarrow$ Pi enters immediately. No indefinite postponement when CS is free.

3. Bounded Waiting:

After Pi sets $\text{flag}[i]=\text{true}$ and $\text{turn}=j$, Pi waits at most one complete entry by Pj before entering. Once Pj exits and sets $\text{flag}[j]=\text{false}$ (or sets $\text{turn}=i$), Pi enters. So waiting is bounded by 1.

Example Trace:

Step	P0 Action	P1 Action	turn	flag[0]	flag[1]
1	flag[0]=true		0	true	false
2	turn=1		1	true	false
3	Checks: $\text{flag}[1]\&\&\text{turn}==1$ $\rightarrow$ false $\rightarrow$ ENTERS CS		1	true	false
4	In CS	flag[1]=true, turn=0	0	true	true
5	In CS	Checks: $\text{flag}[0]\&\&\text{turn}==0 \rightarrow$ true $\rightarrow$ WAITS	0	true	true
6	flag[0]=false, exits CS		0	false	true
7		Condition false $\rightarrow$ ENTERS CS	0	false	true

Q10. Apply the methods — how deadlock can be avoided? (Banker's Algorithm) [10 Marks]

Deadlock avoidance requires that the OS know in advance the maximum resource needs of each process. The system dynamically grants resources only if the resulting state is SAFE.

Safe State: A state is safe if there exists a safe sequence — an ordering of processes such that each process can be allocated its maximum resources using currently available + resources held by preceding processes.

Banker's Algorithm — Example:

5 Processes (P0-P4), 3 Resource Types: A=10, B=5, C=7

Process	Max (A,B,C)	Allocated (A,B,C)	Need (Max-Alloc)	Available (A,B,C)
P0	7,5,3	0,1,0	7,4,3	
P1	3,2,2	2,0,0	1,2,2	
P2	9,0,2	3,0,2	6,0,0	
P3	2,2,2	2,1,1	0,1,1	
P4	4,3,3	0,0,2	4,3,1	3,3,2 (initially)

Running the Banker's Algorithm:

Available = Total - Allocated = (10,5,7) - (7,2,3) = (3,3,2)

22. Work=(3,3,2). Check P1: Need(1,2,2) ≤ Work(3,3,2)? YES → Run P1. Work = (3,3,2)+(2,0,0) = (5,3,2). P1 done.

23. Work=(5,3,2). Check P3: Need(0,1,1) ≤ (5,3,2)? YES → Run P3. Work=(5,3,2)+(2,1,1)=(7,4,3). P3 done.

24. Work=(7,4,3). Check P4: Need(4,3,1) ≤ (7,4,3)? YES → Run P4. Work=(7,4,3)+(0,0,2)=(7,4,5). P4 done.

25. Work=(7,4,5). Check P0: Need(7,4,3) ≤ (7,4,5)? YES → Run P0. Work=(7,4,5)+(0,1,0)=(7,5,5). P0 done.

26. Work=(7,5,5). Check P2: Need(6,0,0) ≤ (7,5,5)? YES → Run P2. P2 done.

Safe Sequence: P1 → P3 → P4 → P0 → P2 ✓ (System is in SAFE STATE)

Resource Request Check: When a process P_i requests resources, Banker's checks:

27. Request ≤ Need[i] (doesn't exceed declared max)

28. Request ≤ Available (resources actually free)

29. Tentatively allocate → run safety algorithm → if SAFE, grant. Else rollback and wait.

Q11. Solve the Readers and Writers Problem using semaphore. [10 Marks]

Problem Statement: Multiple readers can read simultaneously, but a writer needs exclusive access. No reader should be kept waiting unless a writer has access.

Shared Variables:

```
semaphore mutex = 1; // protects read_count
semaphore wrt = 1; // exclusive access for writers
int read_count = 0; // number of active readers
```

Writer Process:

```
do {
    wait(wrt); // get exclusive write lock
    // *** WRITE DATA ***
    signal(wrt); // release write lock
} while(true);
```

Reader Process:

```
do {
    wait(mutex); // protect read_count
    read_count++;
    if (read_count == 1)
        wait(wrt); // first reader locks out writers
```

```

signal(mutex);
// *** READ DATA ***
wait(mutex);
read_count--;
if (read_count == 0)
    signal(wrt); // last reader unlocks writers
signal(mutex);
} while(true);

```

Explanation:

- First reader arriving: sets read_count=1, calls wait(wrt) → blocks any waiting writer.
- Subsequent readers: increment read_count, enter freely (wrt already locked by first reader).
- Last reader leaving: sets read_count=0, calls signal(wrt) → wakes up waiting writer.
- Writers: wait on wrt — granted only when read_count=0 (no active readers).

Semaphore Values Trace:

Event	mutex	wrt	read_count
Initial	1	1	0
R1 enters: wait(mutex), rc=1, wait(wrt)	0→1	0	1
R2 enters: wait(mutex), rc=2, no wait(wrt)	0→1	0	2
R1 exits: rc=1, no signal(wrt)	0→1	0	1
R2 exits: rc=0, signal(wrt)	0→1	1	0
Writer enters: wait(wrt)	1	0	0

Q12. Resource Allocation Graph — uses, deadlock detection, and methods for handling deadlock. [10 Marks]

Resource Allocation Graph (RAG) is a directed graph $G=(V,E)$ where:

- $V = \{P_1, \dots, P_n\} \cup \{R_1, \dots, R_m\}$ — process and resource nodes
- Request edge $P_i \rightarrow R_j$: P_i is requesting an instance of R_j
- Assignment edge $R_j \rightarrow P_i$: one instance of R_j is assigned to P_i
- Resource rectangles contain dots = number of instances

Deadlock Detection with RAG:

- Single instance per resource: Cycle in RAG $\leftrightarrow$ Deadlock exists
- Multiple instances: Cycle is necessary but not sufficient → use resource allocation matrix algorithm

Example RAG with Deadlock:

P1 holds R1, requests R2 | P2 holds R2, requests R1

Edges: $R_1 \rightarrow P_1$, $P_1 \rightarrow R_2$, $R_2 \rightarrow P_2$, $P_2 \rightarrow R_1 \rightarrow$ Cycle $P_1 \rightarrow R_2 \rightarrow P_2 \rightarrow R_1 \rightarrow P_1 \rightarrow$ DEADLOCK

Methods for Handling Deadlock:

1. Deadlock Prevention

- Eliminate one of the 4 Coffman conditions at design time
- Eliminate Hold & Wait: request all resources at once
- Eliminate Circular Wait: impose total ordering on resources

2. Deadlock Avoidance

- Banker's Algorithm — only grant request if resulting state is safe

- Requires advance knowledge of maximum resource needs

3. Deadlock Detection and Recovery

- Allow deadlock to occur; periodically run detection algorithm
- Detection: Find cycle in RAG (single instance) or run resource-request matrix algorithm (multiple instances)
- Recovery methods:
 - a) Process Termination: Kill one or all deadlocked processes (costly — work lost)
 - b) Resource Preemption: Take resource from a process, give to another. Issues: victim selection, rollback, starvation.

4. Ignore the Problem (Ostrich Algorithm)

- Used by most OSes (including Linux, Windows) for simplicity
- Assumes deadlock is rare; let it happen and reboot if needed

Q13. Describe in detail: Address Binding and Swapping. [10 Marks]

A. ADDRESS BINDING

Address binding is the process of mapping logical (program) addresses to physical memory addresses. This can happen at three different stages:

1. Compile Time Binding

- The physical memory address is known at compile time — compiled directly into the object code.
- Program must always load at the same address.
- Advantage: Fast. Disadvantage: Not flexible; must recompile if memory location changes.
- Example: .COM files in MS-DOS

2. Load Time Binding

- If memory location not known at compile time, compiler generates relocatable code.
- Addresses are bound when the program is loaded into memory by the loader.
- If starting address changes, all addresses in the code must be recomputed.

3. Execution Time (Run Time) Binding

- Addresses are bound at execution time. Process can be moved in memory during execution.
- Requires hardware support: base and limit registers, or full MMU with page tables.
- Modern OS uses this — enables virtual memory, relocation, and swapping.

Logical vs Physical Address Space:

- Logical address: Generated by CPU during execution. User sees only logical addresses.
- Physical address: Actual address in RAM. MMU translates logical → physical.
- Physical address = Base register + Logical address (for simple contiguous allocation)

B. SWAPPING

Swapping is moving an entire process from RAM to disk (swap out) and back (swap in).

Purpose: Enable more processes to run than RAM can hold.

Process:

30. Scheduler selects a process to swap out (usually long-waiting or low-priority)
31. OS saves process's memory image to disk (swap space)
32. RAM freed for other processes
33. When needed again: OS swaps process back in (possibly to a different location)
34. Address binding must be execution-time for this to work

Variants:

- Standard Swapping: Entire process moved
- Swapping with Paging: Only some pages moved (more efficient) — used in modern OS

Limitations:

- Slow: Disk I/O is 1000x slower than RAM access
- If process has pending I/O: Cannot swap out safely (DMA may write to wrong memory)

Feature	Compile Time	Load Time	Run Time
When bound	Compile	Load	Execution
Flexibility	None	Low	High
Hardware needed	None	None	MMU/Registers
Modern OS uses	No	No	Yes

Q14. Discuss in detail about the paging and structure of the page table. [10 Marks]

Paging is a non-contiguous memory allocation scheme that eliminates external fragmentation by dividing:

- Physical memory into fixed-size blocks called frames
- Logical memory (process) into same-size blocks called pages
- Pages are loaded into any available frames — contiguity not required

Address Translation:

Logical address = [Page Number (p) | Page Offset (d)]

If logical address space = 2^m and page size = 2^n bytes:

- Page number p uses (m-n) bits
- Page offset d uses n bits
- Physical address = frame_number $\times$ page_size + offset

Example: Page size = 4KB = 2^{12} . Logical address = 12-bit offset + p-bit page number.

Page Table:

Each process has its own page table. Maps page numbers to frame numbers.

Page Number	Frame Number	Valid bit
0	5	1
1	3	1
2	9	0 (not in memory)
3	2	1

Structure of Page Table:

- 1. Single-Level Page Table: Simple array indexed by page number. Problem: Too large for large address spaces (e.g., 32-bit = 1M entries $\times$ 4B = 4MB per process).
- 2. Two-Level (Hierarchical) Page Table: Page table itself is paged. Outer page table + inner page table. Reduces memory for sparse address spaces.
- 3. Inverted Page Table: One entry per physical frame (not per page). Much smaller. Entry = {PID, page number}. Disadvantage: Slow lookup — must search entire table.
- 4. Hashed Page Table: Hash the page number for faster lookup. Used with inverted tables.

TLB (Translation Lookaside Buffer):

- A fast hardware cache of recently used page table entries

- Effective Access Time (EAT) = hit-ratio $\times$ (TLB time + memory time) + (1-hit-ratio) $\times$ (TLB time + 2 $\times$ memory time)
- With TLB hit ratio 98%: EAT $\approx 0.98 \times (20+100) + 0.02 \times (20+200) = 117.6 + 4.4 = 122\text{ns}$

Advantages of Paging:

- No external fragmentation
- Simple allocation — any free frame can hold any page
- Enables virtual memory

Disadvantage: Internal fragmentation (last page of a process may not be full).

Q15. Page Replacement Algorithm — Solved Problem. [10 Marks]

Given: Reference String = 7, 0, 1, 2, 0, 3, 0, 4, 2, 3, 0, 3, 2 | Number of Frames = 3

———— FIFO (First-In First-Out) ————

Replace the page that has been in memory the longest.

Ref	Frames (oldest→newest)	Page Fault?
7	7,-,-	Yes
0	7,0,-	Yes
1	7,0,1	Yes
2	2,0,1 (replace 7)	Yes
0	2,0,1	No (hit)
3	2,3,1 (replace 0)	Yes
0	2,3,0 (replace 1)	Yes
4	4,3,0 (replace 2)	Yes
2	4,2,0 (replace 3)	Yes
3	4,2,3 (replace 0)	Yes
0	0,2,3 (replace 4)	Yes
3	0,2,3	No (hit)
2	0,2,3	No (hit)

FIFO Page Faults = 10

———— LRU (Least Recently Used) ————

Replace the page that has not been used for the longest time.

Ref	Frames	Page Fault?
7	7,-,-	Yes
0	7,0,-	Yes
1	7,0,1	Yes
2	2,0,1 (replace 7, LRU)	Yes
0	2,0,1	No

Ref	Frames	Page Fault?
3	2,0,3 (replace 1, LRU)	Yes
0	2,0,3	No
4	4,0,3 (replace 2, LRU)	Yes
2	4,0,2 (replace 3, LRU)	Yes
3	4,3,2 (replace 0, LRU)	Yes
0	0,3,2 (replace 4, LRU)	Yes
3	0,3,2	No
2	0,3,2	No

LRU Page Faults = 8

———— Optimal Page Replacement ————

Replace the page that will NOT be used for the longest time in the future.

Ref	Frames	Page Fault?
7	7,-,-	Yes
0	7,0,-	Yes
1	7,0,1	Yes
2	7,0,2 (replace 1 — next use far)	Yes
0	7,0,2	No
3	3,0,2 (replace 7 — not used again)	Yes
0	3,0,2	No
4	4,0,2 (replace 3 — next use far)	Yes
2	4,0,2	No
3	4,3,2 (replace 0 — next use far)	Yes
0	0,3,2 (replace 4 — not used again)	Yes
3	0,3,2	No
2	0,3,2	No

Optimal Page Faults = 7 (minimum possible — used as benchmark)

Algorithm	Page Faults	Remarks
FIFO	10	Simple but Belady's anomaly possible
LRU	8	Good approximation, expensive to implement
Optimal	7	Best possible — theoretical benchmark

Q16. Explain copy-on-write, frame allocation, and thrashing. How does the OS handle thrashing? [10 Marks]

1. Copy-on-Write (COW)

When a process calls `fork()`, instead of immediately copying the parent's entire address space, the OS marks all pages as shared (read-only). A copy of a page is made only when either process tries to WRITE to it.

- Benefit: Saves memory and time when child immediately calls `exec()` (common pattern).
- Only modified pages need to be copied — rest remain shared.
- Used by: Linux, macOS, Windows fork-like mechanisms.
- Example: Parent has 100 pages. After `fork()`, if child writes to page 5 only, only page 5 is copied — 99 pages remain shared.

2. Frame Allocation

When physical memory is divided into frames and multiple processes compete for them, the OS must decide how many frames to give each process.

- Equal Allocation: Divide frames equally among all processes. Simple but ignores process size.
- Proportional Allocation: Allocate frames proportional to process size. S_i = size of process i . Frames for $i = (S_i / \sum S) \times \text{total frames}$.
- Priority Allocation: Larger processes or higher-priority processes get more frames.
- Global Replacement: Process can take frames from other processes — higher throughput but less predictable.
- Local Replacement: Process can only replace from its own frames — more predictable but underutilizes free frames.

3. Thrashing

Thrashing occurs when a process spends more time swapping pages in and out than actually executing. CPU utilization drops dramatically as processes constantly fault.

Cause: Too many processes competing for too few frames. Each process has fewer frames than its working set — constantly page faults.

Working Set: The set of pages a process is actively using during a time window Δ .

How OS Handles Thrashing:

- Working Set Model: OS tracks working set (WS_i) of each process. Total demand $D = \sum WS_i$. If $D >$ total frames $\rightarrow$ thrashing $\rightarrow$ suspend a process to free frames.
- Page Fault Frequency (PFF) Control: If a process's page fault rate $>$ upper threshold $\rightarrow$ give it more frames. If $<$ lower threshold $\rightarrow$ take frames away. If no frames to give $\rightarrow$ suspend a process.
- Reduce Degree of Multiprogramming: Swap out one or more processes entirely — gives remaining processes enough frames.
- Increase RAM: Long-term solution.

Q18. Explain the roles of the loader and linker. Illustrate with a scenario. [10 Marks]

Scenario: User writes a C program that uses `printf()` and runs it.

Step 1: Source Code $\rightarrow$ Compiler $\rightarrow$ Object Files

The C compiler converts source code (`hello.c`) into machine code object files (`hello.o`). These contain:

- Machine code instructions
- Symbol table: list of defined and undefined symbols (e.g., `printf` is undefined in `hello.o`)
- Relocation information: addresses that need to be filled in later

Step 2: The Linker

The linker (`ld`) combines multiple object files and libraries into a single executable.

Tasks of the Linker:

- 1. Symbol Resolution: Finds all undefined symbols (printf defined in libc.a) and connects them.
- 2. Relocation: Adjusts all addresses relative to the final load address.
- 3. Produces: A single executable file (a.out / hello.exe) with all code combined.

Types of Linking:

- Static Linking: All library code copied into executable at link time. Executable is self-contained but large.
- Dynamic Linking: Library code NOT copied. Executable contains references. Actual linking happens at load/run time (uses .so or .dll files). Saves disk space, allows library updates.

Step 3: The Loader

When user runs the program, the OS invokes the loader.

Tasks of the Loader:

- 1. Reads the executable file from disk
- 2. Allocates memory for the process (text, data, stack, heap segments)
- 3. Copies the program image from disk to memory
- 4. Performs final address relocation if needed (for relocating loader)
- 5. Sets up the stack with command-line arguments
- 6. Transfers control to the program's entry point (main())

Types of Loaders:

- Absolute Loader: Loads program at a fixed address — simple but inflexible.
- Relocating Loader: Can load program at any address; adjusts addresses at load time.
- Dynamic Loader: Loads parts of program on demand — used for dynamic libraries.

Stage	Who does it	Input	Output
Compile	Compiler (gcc -c)	Source (.c)	Object file (.o)
Link	Linker (ld)	Object files + libraries	Executable (a.out)
Load	OS Loader	Executable	Process in memory
Execute	CPU	Process in memory	Program output

Q21. Explain the differences between processes and threads. [10 Marks]

Feature	Process	Thread
Definition	A program in execution with its own address space	A unit of execution within a process
Address Space	Separate — each process has its own	Shared — threads share process memory
Resources	Own copy of code, data, heap, stack, files, PCB	Shared code, data, heap; own stack, PC, registers
Creation overhead	High — full copy of parent (fork)	Low — just stack and registers allocated
Context switch	Expensive — must switch page tables, TLB flush	Fast — no address space change
Communication	IPC (pipes, sockets, shared memory)	Direct via shared memory — no IPC needed
Isolation	Isolated — one crash doesn't	Not isolated — one thread crash

Feature	Process	Thread
	affect others	kills entire process
Synchronization	Simpler — no shared memory by default	Required — race conditions on shared data
Example	Chrome opens each tab as a process	Web server handles each request in a thread

Benefits of Threads over Processes:

- Responsiveness: A multithreaded program can continue executing even if one thread is blocked.
- Resource sharing: Threads share memory — no need for expensive IPC.
- Economy: Creating and switching threads is much cheaper than processes.
- Scalability: Threads can run in parallel on multiple cores.

Q22. Illustrate with examples how multithreading models handle thread management. [10 Marks]

Multithreading models define how user-level threads (ULTs) are mapped to kernel-level threads (KLTs).

1. Many-to-One Model (User-Level Threads)

- Many user threads → mapped to ONE kernel thread
- Thread management done entirely in user space by a thread library
- Advantage: Thread operations very fast (no kernel call for thread switch)
- Disadvantage: One thread blocking (e.g., I/O) → blocks ENTIRE process. Cannot use multiple cores.
- Example: GNU Portable Threads library

2. One-to-One Model (Kernel-Level Threads)

- Each user thread → one kernel thread
- True parallelism on multiprocessor systems
- Advantage: One thread blocking doesn't affect others. Full multicore parallelism.
- Disadvantage: Creating a user thread creates a kernel thread — expensive. Systems limit number of threads.
- Example: Windows, Linux (pthreads on Linux are 1:1)

3. Many-to-Many Model (Hybrid / M:N Model)

- Many user threads → mapped to many kernel threads ($\leq$ user threads)
- OS creates enough kernel threads to match number of CPUs
- Advantage: Best of both — no restriction on user threads, true parallelism possible, blocking one thread doesn't block others.
- Disadvantage: Complex to implement.
- Example: Solaris (up to version 9), Windows with fiber package

4. Two-Level Model

- Variant of M:N: allows binding a specific user thread to a specific kernel thread for critical threads.
- Example: IRIX, HP-UX, Tru64 UNIX

Model	Mapping	Parallelism	Blocking	Example
Many-to-One	M ULT → 1 KLT	No	Yes — whole process	GNU Portable Threads
One-to-One	1 ULT → 1 KLT	Yes	No	Linux, Windows
Many-to-Many	M ULT → N KLT	Yes	No	Solaris (old)

Model	Mapping	Parallelism	Blocking	Example
	($N \leq M$)			
Two-Level	M:N + pinned threads	Yes	No	HP-UX, IRIX

Q23. Discuss various IPC mechanisms. How does message passing differ from shared memory?
[10 Marks]

IPC mechanisms allow processes to communicate and synchronize. Main mechanisms:

1. Pipes

- Unidirectional channel between two related processes (parent-child).
- Ordinary pipes: require parent-child relationship. Named pipes (FIFOs): allow unrelated processes.
- Example: `ls | grep .c` (output of `ls` is piped to `grep`)

2. Message Queues

- Messages stored in kernel queue. Processes send/receive messages independently of timing.
- Messages have types. Multiple readers/writers supported.
- Advantage over pipes: messages have boundaries; random access by type.

3. Shared Memory

- A region of RAM shared between processes (attached via `shmget/shmat` in POSIX).
- Fastest IPC — direct memory access without kernel involvement.
- Requires synchronization (semaphores) to avoid race conditions.

4. Semaphores

- Not for data transfer — used purely for synchronization between processes.
- Binary semaphore (mutex) or counting semaphore.

5. Sockets

- Communication endpoint. Supports local (Unix domain sockets) and network (TCP/UDP) communication.
- Used by networked applications.

6. Signals

- Asynchronous notification sent to a process (e.g., `SIGKILL`, `SIGTERM`, `SIGCHLD`).
- Limited data — just the signal type. Used for event notification.

Feature	Shared Memory	Message Passing
Speed	Fastest — direct RAM access	Slower — kernel involved each msg
Synchronization	Must be done explicitly	Built-in (queued messages)
Data volume	Any size (limited by RAM)	Usually small messages
Ease	Harder — programmer manages sync	Easier — well-defined send/receive
Works across network	No (same machine only)	Yes (sockets, MPI)
OS involvement	Only at setup (<code>shmget/shmat</code>)	Every send/receive is a syscall
Example	POSIX shm, mmap	Pipes, message queues, sockets

Q24. FCFS and Round Robin Scheduling — Solved Problem. [10 Marks]

Given Processes (TQ = 2ms for RR):

Process	AT	BT
P1	0	5
P2	1	3
P3	2	8
P4	3	6

———— FCFS ————

Process	AT	BT	Start	Finish	TAT	WT
P1	0	5	0	5	5	0
P2	1	3	5	8	7	4
P3	2	8	8	16	14	6
P4	3	6	16	22	19	13

Gantt: | P1(0-5) | P2(5-8) | P3(8-16) | P4(16-22) |

Avg TAT = $(5+7+14+19)/4 = 45/4 = 11.25 \text{ ms}$ | Avg WT = $(0+4+6+13)/4 = 23/4 = 5.75 \text{ ms}$

———— Round Robin (TQ=2) ————

Ready queue order tracked by arrival time. Remaining burst times shown.

Time	Process	Remaining (P1 P2 P3 P4)	Event
0-2	P1	3 3 8 6	P2,P3 arrive during run
2-4	P2	3 1 8 6	P3,P4 arrive; P1 goes to back
4-6	P3	3 1 6 6	P2 has 1ms left
6-8	P4	3 1 4 4	
8-11	P1	0 1 4 4	P1 finishes at 11
11-12	P2	0 0 4 4	P2 finishes at 12
12-14	P3	0 0 2 4	
14-16	P4	0 0 2 2	
16-18	P3	0 0 0 2	P3 finishes at 18
18-20	P4	0 0 0 0	P4 finishes at 20

Gantt: | P1(0-2)|P2(2-4)|P3(4-6)|P4(6-8)|P1(8-11)|P2(11-12)|P3(12-14)|P4(14-16)|P3(16-18)|P4(18-20) |

Process	AT	BT	Finish	TAT	WT
P1	0	5	11	11	6
P2	1	3	12	11	8
P3	2	8	18	16	8

Process	AT	BT	Finish	TAT	WT
P4	3	6	20	17	11

$\text{Avg TAT} = (11+11+16+17)/4 = 55/4 = 13.75 \text{ ms}$ | $\text{Avg WT} = (6+8+8+11)/4 = 33/4 = 8.25 \text{ ms}$

Q25. Priority and Non-Preemptive SJF Scheduling — Solved Problem. [10 Marks]

Given (lower number = higher priority):

Process	AT	BT	Priority
P1	0	10	3
P2	1	5	1
P3	2	8	4
P4	3	4	2

———— Non-Preemptive Priority ————

At t=0: Only P1 available → P1 starts (BT=10), finishes at t=10.

At t=10: P2(pri=1), P3(pri=4), P4(pri=2). Highest priority = lowest number = P2.

At t=15: P3(pri=4), P4(pri=2). P4 runs next (priority 2 > 4).

At t=19: P3 runs.

Process	AT	BT	Priority	Start	Finish	TAT	WT
P1	0	10	3	0	10	10	0
P2	1	5	1	10	15	14	9
P4	3	4	2	15	19	16	12
P3	2	8	4	19	27	25	17

Gantt: | P1(0-10) | P2(10-15) | P4(15-19) | P3(19-27) |

$\text{Avg TAT} = (10+14+16+25)/4 = 65/4 = 16.25 \text{ ms}$ | $\text{Avg WT} = (0+9+12+17)/4 = 38/4 = 9.5 \text{ ms}$

———— Non-Preemptive SJF ————

At t=0: Only P1 available → P1 runs. Finishes at t=10.

At t=10: P2(BT=5), P3(BT=8), P4(BT=4). Shortest = P4.

At t=14: P2(BT=5), P3(BT=8). Shortest = P2.

At t=19: P3 runs.

Process	AT	BT	Start	Finish	TAT	WT
P1	0	10	0	10	10	0
P4	3	4	10	14	11	7
P2	1	5	14	19	18	13
P3	2	8	19	27	25	17

Gantt: | P1(0-10) | P4(10-14) | P2(14-19) | P3(19-27) |

Avg TAT = $(10+11+18+25)/4 = 64/4 = 16 \text{ ms}$ | Avg WT = $(0+7+13+17)/4 = 37/4 = 9.25 \text{ ms}$

Q26. Explain Peterson's solution — mutual exclusion, progress, bounded waiting with example.

[10 Marks]

(See also B9 for algorithm. This is the detailed 10-mark version.)

Peterson's solution solves the critical section problem for 2 processes using only shared variables — no special hardware instructions.

Shared Variables:

```
int turn;          // 0 or 1 — whose turn to enter
boolean flag[2];   // flag[i] = Pi wants to enter CS
```

Full Algorithm for Pi (where j = 1 - i):

```
do {
    flag[i] = true;          // declare: I want to enter
    turn = j;                // give courtesy to other
    while (flag[j] && turn == j); // busy wait
    /* ===== CRITICAL SECTION ===== */
    flag[i] = false;         // leaving CS
    /* ===== REMAINDER SECTION ===== */
} while (true);
```

1. Mutual Exclusion — Proof:

Assume both P0 and P1 are in CS simultaneously.

- P0 in CS: it exited while loop $\rightarrow$ either $\text{flag}[1]=\text{false}$ OR $\text{turn}=0$
- P1 in CS: it exited while loop $\rightarrow$ either $\text{flag}[0]=\text{false}$ OR $\text{turn}=1$
- Since both are in CS: $\text{flag}[0]=\text{true}$ AND $\text{flag}[1]=\text{true}$.
- If $\text{turn}=0$: P1 waited at while $(\text{flag}[0] \& \& \text{turn}==0) \rightarrow \text{true} \rightarrow$ P1 should still be waiting. CONTRADICTION.
- If $\text{turn}=1$: P0 waited at while $(\text{flag}[1] \& \& \text{turn}==1) \rightarrow \text{true} \rightarrow$ P0 should still be waiting. CONTRADICTION.
- Both cases: contradiction $\rightarrow$ Mutual Exclusion guaranteed.

2. Progress — Proof:

- If Pi wants to enter ($\text{flag}[i]=\text{true}$) and Pj doesn't ($\text{flag}[j]=\text{false}$):
- Pi's while condition: $\text{flag}[j] \& \& \text{turn}==j = \text{false} \& \& _ = \text{false} \rightarrow$ Pi enters immediately.
- If both want to enter: one has $\text{turn}=i$, so it enters. No indefinite postponement.

3. Bounded Waiting — Proof:

- After Pi sets $\text{flag}[i]=\text{true}$ and $\text{turn}=j$:
- If Pj is in CS, Pj will eventually exit and set $\text{flag}[j]=\text{false}$ OR set $\text{turn}=i$ (when Pj re-enters).
- Pi then exits while loop and enters CS.
- Pj enters CS at most once before Pi gets in $\rightarrow$ bounded waiting of 1.

Detailed Example Trace (P0 and P1 competing):

Step	P0	P1	turn	flag[0]	flag[1]	CS?
1	flag[0]=true	—	—	T	F	—
2	turn=1	—	1	T	F	—
3	while: F $\rightarrow$ skip	flag[1]=true	1	T	T	—

Step	P0	P1	turn	flag[0]	flag[1]	CS?
4	P0 ENTERS CS	turn=0	0	T	T	P0
5	In CS	while: T&&T=T→wait	0	T	T	P0
6	flag[0]=false	still waiting	0	F	T	—
7	Remainder	while: F&&_F=enter	0	F	T	P1
8	—	P1 ENTERS CS	0	F	T	P1

Q27. Given 5 processes and 5 resources, construct a RAG, detect deadlock, suggest recovery. [10 Marks]

Scenario Setup:

Processes: P1, P2, P3, P4, P5 | Resources: R1(1 inst), R2(2 inst), R3(1 inst), R4(1 inst), R5(2 inst)

Process	Holds	Requests
P1	R1	R2
P2	R2 (1 inst), R3	R1
P3	R4	R2 (1 inst)
P4	R5 (1 inst)	R3
P5	R5 (1 inst)	R4

RAG Edges:

- Assignment: R1→P1, R2→P2, R3→P2, R4→P3, R5→P4, R5→P5
- Request: P1→R2, P2→R1, P3→R2, P4→R3, P5→R4

Deadlock Detection:

Trace cycles:

- P1 requests R2 → R2 held by P2 → P2 requests R1 → R1 held by P1 → Cycle: P1→R2→P2→R1→P1 → DEADLOCK between P1 and P2
- P4 requests R3 → R3 held by P2 → P2 requests R1 → R1 held by P1 → P1 requests R2 → R2 held by P2 → contributes to deadlock chain
- P5 requests R4 → R4 held by P3 → P3 requests R2 → R2 held by P2 → P2 in deadlock chain → extended deadlock

Deadlock exists. Deadlocked processes: P1, P2, P3, P4, P5 (all involved).

Recovery Methods:

- 1. Process Termination: Terminate P2 (holding R2 and R3). R2 released → P1 or P3 can proceed. R3 released → P4 can proceed. Then terminate or resolve remaining.
- 2. Resource Preemption: Preempt R1 from P1 → give to P2 → P2 completes → releases R2,R3 → P1, P3, P4 proceed. Must rollback P1 to a safe checkpoint (roll-back cost).
- 3. Starvation concern: Don't always pick same victim for preemption — use cost-based victim selection (choose process with minimum rollback cost).

Q28. Differentiate Deadlock Prevention and Deadlock Avoidance. Explain with examples. [10 Marks]

Marks]

Both aim to ensure deadlock never occurs, but use different strategies:

Feature	Deadlock Prevention	Deadlock Avoidance
Strategy	Eliminate ≥ 1 Coffman condition at design time	Allow conditions; dynamically check each request
When applied	Design/compile time	Run time (per resource request)
Knowledge needed	Structure of system only	Max future resource needs of each process
Overhead	Low at runtime	High — safety check per request
Utilization	Lower — more requests denied	Higher — only unsafe requests denied
Example algorithm	Resource ordering	Banker's Algorithm

Deadlock Prevention — Techniques:

- 1. Deny Mutual Exclusion: Make resources shareable (e.g., read-only files). Cannot work for inherently non-shareable resources (printers, tape drives).
- 2. Deny Hold and Wait: Process must request all needed resources before starting. OR: process must release all resources before requesting new ones.

Example: Process needs R1 and R2. Must request both at start — cannot request R1, then later request R2.

- 3. Allow Preemption: If P_i requests resource not available, it releases all held resources. Resources added to preempted list. P_i restarts when all available.
- 4. Deny Circular Wait: Define total ordering of resource types ($R_1 < R_2 < \dots < R_m$). Processes must request resources in increasing order.

Example: Process can request R3 only after R1 and R2. This prevents circular wait.

Deadlock Avoidance — Banker's Algorithm:

- System is in SAFE state if there is a safe sequence: $P_1, P_2, \dots, P_n$ where each P_i can obtain needed resources from available + resources held by preceding processes.
- On each resource request: tentatively allocate $\rightarrow$ run safety algorithm $\rightarrow$ if safe: grant $\rightarrow$ if unsafe: deny (process waits).

Example:

Available = [3,3,2]. P_0 requests [1,0,2].

35. Check request $\leq$ Need[P_0] and request $\leq$ Available $\rightarrow$ Yes

36. Tentatively: Available=[2,3,0], Allocated[P_0] += [1,0,2]

37. Run safety algorithm $\rightarrow$ safe sequence exists $\rightarrow$ GRANT request

Q29-30. Readers-Writers Problem using Semaphores — full solution with logic. [10 Marks]

Problem: A database shared among readers and writers. Readers can read simultaneously. Writers need exclusive access. No reader waits unless a writer has access.

Semaphore Initialization:

```
semaphore mutex = 1; // protects read_count (binary semaphore)
semaphore wrt = 1; // controls write access (binary semaphore)
int read_count = 0; // tracks number of active readers
```

Why these values: mutex=1 means it's free (unlocked). wrt=1 means no writer active.

Writer Process Code:

```

do {
    wait(wrt);    // wait until no reader or writer active
    // *** CRITICAL SECTION: WRITE ***
    // Modify shared data here
    signal(wrt);  // release: allow readers/writers in
} while (true);

```

Reader Process Code:

```

do {
    // ENTRY SECTION
    wait(mutex);    // lock: updating read_count
    read_count++;
    if (read_count == 1)    // first reader?
        wait(wrt);    // lock out writers
    signal(mutex);    // unlock read_count
    // *** CRITICAL SECTION: READ ***
    // Read shared data here
    // EXIT SECTION
    wait(mutex);    // lock: updating read_count
    read_count--;
    if (read_count == 0)    // last reader?
        signal(wrt);    // allow writers in
    signal(mutex);    // unlock read_count
} while (true);

```

Semaphore Logic Explanation:

- mutex semaphore: Ensures only one reader updates read_count at a time — without mutex, two readers could both see read_count=0 and both call wait(wrt) — double lock.
- wrt semaphore: The 'write lock.' First reader locks it, last reader unlocks it. Writer locks it exclusively.
- read_count: Tracks active readers. Only first reader blocks writers; only last reader wakes writers.

Correctness Properties:

- Mutual Exclusion for Writers: wait(wrt) ensures only one writer at a time, and no readers while writing.
- Multiple Readers: Since readers don't call wait(wrt) directly (only first one does), multiple readers can read simultaneously.
- No Reader Starvation (1st Readers-Writers): Writers may starve if readers keep arriving. Solution requires additional mechanisms (2nd RW problem).

Scenario	mutex	wrt	read_count	Outcome
R1 enters (first)	0→1	0 (locked)	1	Writers blocked
R2 enters	0→1	0	2	Both reading
W1 arrives	—	0 (busy wait)	2	Writer waits
R1 exits	0→1	0	1	Writer still waits
R2 exits (last)	0→1	1 (unlocked)	0	Writer enters
W1 writes	—	0	0	Exclusive access

Q31. Explain logical and physical address space. How does MMU translate between them? [10 Marks]

Logical Address Space

- Set of all logical (virtual) addresses generated by the CPU for a process.
- Each process has its own logical address space starting from 0.
- Logical addresses are what the programmer and compiler see — they do NOT correspond to actual RAM locations.
- Range: 0 to (process size - 1)

Physical Address Space

- Set of all actual physical memory addresses (RAM locations).
- Shared by all processes — each occupies a different region.
- Determined by MMU at run time.
- Range: 0 to (RAM size - 1)

Memory Management Unit (MMU)

The MMU is hardware that dynamically translates logical addresses to physical addresses at run time.

Simple MMU (Base Register Scheme):

- Base register holds the starting physical address of the process.
- Physical address = base + logical address
- Limit register ensures logical address < limit (else trap).

Example: base=14000, limit=3000. Logical address 346 → Physical 14346. Logical address 3500 → TRAP ($\geq$ limit).

MMU with Paging:

- Logical address split into: [Page Number p | Offset d]
- Page table maps p → frame number f
- Physical address = $f \times \text{page_size} + d$

Example: Page size=4KB. Logical=0x3A5 = page 0, offset 0x3A5. Page 0 → frame 5. Physical = $5 \times 4096 + 0x3A5 = 20992 + 933 = 21925$.

TLB (Translation Lookaside Buffer):

- A fast cache inside MMU storing recently used page → frame mappings.
- TLB Hit: Address found → fast translation (1 memory access)
- TLB Miss: Must access page table in RAM (2 memory accesses) → update TLB
- Effective Access Time = $\text{hit_rate} \times (\text{TLB_time} + \text{mem_time}) + (1 - \text{hit_rate}) \times (\text{TLB_time} + 2 \times \text{mem_time})$

Address Type	Generated By	Seen By	Example
Logical	CPU	User program	0 to max process size
Physical	MMU	RAM hardware	Actual RAM location
Virtual	Synonym for logical	Same as logical	Same

Q32. Differentiate contiguous and non-contiguous allocation. Explain segmentation. [10 Marks]

Feature	Contiguous Allocation	Non-Contiguous Allocation
Memory layout	Process occupies consecutive memory locations	Process pages/segments scattered in memory
External fragmentation	Yes — wasted gaps between	No (paging) / Reduced

Feature	Contiguous Allocation	Non-Contiguous Allocation
	allocations	(segmentation)
Address translation	Simple — base + offset	Requires page/segment table lookup
Flexibility	Low	High
Example	Fixed/Variable partitioning	Paging, Segmentation

Segmentation

Segmentation divides a program into logical units called segments (e.g., code, stack, heap, data) — each of a different size. Unlike paging, segments correspond to logical program structure.

Logical Address in Segmentation: [Segment Number s | Offset d]

Segment Table: Each entry has:

- Base: physical starting address of segment
- Limit: size of segment

Address Translation:

38. Extract segment number s and offset d from logical address.

39. Look up segment table entry for s → get base and limit.

40. If $d \geq \text{limit}$ → TRAP (protection violation).

41. Physical address = base + d .

Example: Process has 3 segments:

Segment	Base	Limit
0 (Code)	1400	1000
1 (Data)	3200	400
2 (Stack)	4300	100

Logical address (1, 53) → $d=53 < \text{limit}=400$ → Physical = $3200+53 = 3253$

Advantages of Segmentation:

- Matches programmer's logical view of memory
- Easy sharing (e.g., code segment shared between processes)
- Easy protection (each segment has own permissions)
- No internal fragmentation

Disadvantage: External fragmentation (variable-size segments).

Segmentation + Paging: Segments are further paged — eliminates external fragmentation.

Q33. FIFO Page Replacement Algorithm — Solved Problem. [10 Marks]

Given: Reference String = 1,2,3,4,1,2,5,1,2,3,4,5 | Frames = 3

Ref	Frame 1	Frame 2	Frame 3	Fault?
1	1	-	-	Yes
2	1	2	-	Yes
3	1	2	3	Yes

Ref	Frame 1	Frame 2	Frame 3	Fault?
4	4	2	3 (replace 1 — oldest)	Yes
1	4	1	3 (replace 2)	Yes
2	4	1	2 (replace 3)	Yes
5	5	1	2 (replace 4)	Yes
1	5	1	2	No (hit)
2	5	1	2	No (hit)
3	5	3	2 (replace 1)	Yes
4	5	3	4 (replace 2)	Yes
5	5	3	4	No (hit)

Total Page Faults = 9

Belady's Anomaly: FIFO can have MORE page faults with MORE frames (observed in some reference strings).

Q34. Explain Virtual Memory and how demand paging handles a page fault. [10 Marks]

Virtual Memory

Virtual memory is a memory management technique that gives each process the illusion of having a large, contiguous address space — even larger than physical RAM. Only parts of the process actually in use need to be in RAM.

- Benefit 1: Programs can be larger than physical RAM
- Benefit 2: More processes can run concurrently — only active pages loaded
- Benefit 3: Faster process startup — don't need to load entire program
- Implementation: Demand paging (most common), demand segmentation

Demand Paging

Pages are loaded into RAM only when they are actually accessed (demanded), not all at once at process start. Non-resident pages are on disk in the swap space.

Page Table Entry includes a valid bit:

- Valid bit = 1: Page is in physical memory — direct translation
- Valid bit = 0: Page is NOT in memory — page fault occurs

Detailed Page Fault Handling Steps:

42. Process accesses a logical address. CPU sends to MMU.
43. MMU checks page table. Valid bit = 0 → hardware raises a PAGE FAULT TRAP.
44. CPU switches to kernel mode. OS page fault handler invoked.
45. OS checks if the reference is valid (address within process's logical space). If invalid → kill process. If valid → continue.
46. OS finds a free frame in physical memory. If no free frame: run page replacement algorithm (FIFO/LRU/Optimal) to select a victim frame.
47. If victim frame is dirty (modified = 1): write it back to disk (swap out). This requires disk I/O.
48. OS reads the needed page from disk (swap space) into the free frame (swap in).
49. Page table entry updated: frame number set, valid bit set to 1.

50. Blocked process moved to ready queue.

51. CPU restarts the faulting instruction. This time MMU finds valid bit = 1 → successful translation → execution continues.

Performance — Effective Access Time with Demand Paging:

$$EAT = (1-p) \times \text{memory_access} + p \times (\text{page_fault_service_time})$$

Page fault service time $\approx$ 1ms to 10ms (disk access)

Memory access $\approx$ 100ns

Even 1 page fault per 1000 accesses doubles EAT.

Q35. Draw a neat diagram and explain dual mode operation. [10 Marks]

Dual mode operation is a hardware feature that allows the OS to protect itself and other processes by distinguishing between two modes of CPU operation.

Two Modes:

1. User Mode

- Mode bit = 1
- User applications (browsers, games, word processors) run here
- Restricted: cannot execute privileged instructions (e.g., halt CPU, modify page tables, access I/O ports directly)
- If a privileged instruction is attempted → hardware trap (protection fault)

2. Kernel Mode (Supervisor / Privileged Mode)

- Mode bit = 0
- OS kernel, interrupt handlers, device drivers run here
- Full access: any instruction, any memory location
- Mode entry: interrupt, trap, system call

Mode Switching:

Trigger	Direction	Cause
System call (e.g., read())	User → Kernel	Program needs OS service
Interrupt (e.g., timer, I/O)	User → Kernel	Hardware event
Exception / Trap	User → Kernel	Error or intentional (divide-by-zero, page fault)
Return from kernel (iret)	Kernel → User	OS finished servicing request

Dual Mode Sequence (System Call Example):

52. User app calls write() → library issues SYSCALL instruction
53. Hardware: mode bit 0 → 1 (kernel mode), saves user state, jumps to kernel
54. Kernel: validates system call, executes write to file descriptor
55. Kernel: prepares return value, issues return-from-interrupt
56. Hardware: mode bit 1 → 0 (user mode), restores user state
57. User app: continues from after write() call with return value

Why dual mode matters:

- Without it, a user program could modify OS data structures, kill other processes, or access any hardware directly.
- Dual mode ensures only trusted (kernel) code runs privileged operations.

Q36. What are system calls? Explain different categories with examples. [10 Marks]

(Extended version of Part-A Q11. See B2 for full type-by-type explanation.)

Categories with detailed examples:

1. Process Control

```
pid = fork();      // create child process
exec('/bin/ls', args); // replace process image
exit(0);           // terminate process
wait(&status);     // wait for child to finish
getpid();          // get current process ID
```

2. File Management

```
fd = open('file.txt', O_RDONLY);
n = read(fd, buffer, 1024);
write(fd, data, len);
close(fd);
unlink('file.txt'); // delete file
```

3. Device Management

```
fd = open('/dev/sda', O_RDWR); // open device
ioctl(fd, TIOCGWINSZ, &ws);   // get terminal window size
read(fd, buffer, 512);        // read from device
```

4. Information Maintenance

```
gettimeofday(&tv, NULL); // get current time
getpid();                // process ID
sysinfo(&info);           // system information
```

5. Communication

```
pipe(fds);              // create pipe
sock = socket(AF_INET, SOCK_STREAM, 0); // create socket
connect(sock, &addr, sizeof(addr));
send(sock, msg, len, 0);
recv(sock, buf, 1024, 0);
```

6. Protection

```
chmod('secret.txt', 0600); // owner r/w only
chown('file', uid, gid);   // change ownership
umask(022);                // set default permissions
```

Q37. Compare microkernel approach and layered kernel approach. [10 Marks]

Feature	Layered Kernel	Microkernel
Structure	OS divided into N layers (0=hardware, N=user)	Tiny kernel; most services in user space
Kernel size	Medium — all OS services in kernel, just organized in layers	Minimal — IPC, basic scheduling, memory only
Communication	Layer-to-layer function calls (downward only)	Message passing between user-space services

Feature	Layered Kernel	Microkernel
Performance	Good — direct calls within kernel	Overhead — messages cross user-kernel boundary
Reliability	Medium — kernel crash = full crash	High — service crash doesn't kill kernel
Extensibility	Difficult — must add new layer	Easy — add new service in user space
Debugging	Easy — test layer by layer	Moderate — message tracing needed
Example	THE OS, Windows NT (early)	Minix, QNX, L4, macOS Darwin

Layered Structure Detail:

- Layer 0: Hardware
- Layer 1: CPU scheduling
- Layer 2: Memory management
- Layer 3: I/O management
- Layer 4: User programs
- Each layer uses ONLY the services of the layer directly below it.

Microkernel Detail:

- Kernel provides ONLY: IPC, minimal process scheduling, basic memory protection
- File system, device drivers, network stack run as user-space servers
- User app → message to file server → file server reads disk → reply to app
- Benefit: If file server crashes, only restart it — kernel and other services unaffected

Q38. Compare the working of relocating loader and dynamic loader. [10 Marks]

Feature	Relocating Loader	Dynamic Loader
When addresses bound	At load time	At execution time (on demand)
Flexibility	Can load at any memory address	Can load modules as needed during execution
Memory usage	Entire program loaded at once	Only called modules loaded
Speed	Slow start (relocation overhead)	Fast initial load; pay cost on first call
Library linking	All code in executable already	Libraries loaded from .so/.dll files at runtime
Use case	Standard program loading	Dynamic libraries, plugins, overlays

Relocating Loader:

58. Read executable from disk
59. Find relocation table in executable
60. For each relocatable address: add load-start address to convert to correct physical address
61. Copy relocated code/data into memory
62. Transfer control to entry point

Example: Program compiled to start at 0x0. Loader decides to load at 0x4000. All addresses (jump targets, data refs) incremented by 0x4000.

Dynamic Loader (used with dynamic linking):

63. Executable loaded quickly — contains only stubs for dynamic library calls
64. At runtime, when a stub is first called → dynamic linker invoked
65. Dynamic linker finds the .so/.dll file on disk
66. Maps library into process address space
67. Stub updated to point directly to library function (lazy binding)
68. Subsequent calls go directly — no more dynamic linker overhead

Example: printf is in libc.so. First call to printf → dynamic linker loads libc.so into process. All later calls are direct.

Q39. What is IPC? Explain different models with diagrams. [10 Marks]

(See B6 and B23 for detailed coverage. This covers the two fundamental models.)

IPC enables cooperating processes to exchange information. Two fundamental models:

Model 1: Shared Memory

Setup: Producer and Consumer processes communicate via a shared memory region.

```
// Producer creates shared memory:
shmctl = shmget(KEY, SIZE, IPC_CREAT|0666);
shm = (char*)shmat(shmctl, NULL, 0);
// Write to shared memory:
sprintf(shm, 'Hello from Producer!');
// Consumer reads:
shmctl = shmget(KEY, SIZE, 0666);
shm = (char*)shmat(shmctl, NULL, 0);
printf('%s', shm);
```

Buffer Problem (Producer-Consumer with Shared Memory):

- Shared buffer of size N between producer and writer
- in pointer: next empty slot (producer writes here)
- out pointer: next full slot (consumer reads here)
- Full when $(in+1) \% N == out$ | Empty when $in == out$

Model 2: Message Passing

Processes communicate by send/receive — no shared memory needed.

```
// Direct communication:
send(P2, message); // P1 sends to P2
receive(P1, &message); // P2 receives from P1
// Via mailbox (indirect):
send(mailbox_A, message);
receive(mailbox_A, &message);
```

Pipe Example:

```
int fd[2]; pipe(fd);
if (fork() == 0) { // child
    close(fd[1]); // close write end
    read(fd[0], buf, 100); // read from pipe
} else { // parent
```

```

close(fd[0]);    // close read end
write(fd[1], msg, len); // write to pipe
}

```

Q41. Preemptive Priority Scheduling and FCFS — Solved Problem. [10 Marks]

Given (lower number = higher priority):

Process	AT	BT	Priority
P1	0	8	3
P2	1	4	1
P3	2	9	4
P4	3	5	2

———— Preemptive Priority Scheduling ————

P1 starts at t=0 (only process). At t=1, P2 arrives with priority $1 < 3$ (higher) → PREEMPT P1.

P2 runs from t=1. At t=3, P4 arrives with priority $2 > 1$ (lower). No preemption.

P2 finishes at t=5. At t=5: P1(remaining=7, pri=3), P3(pri=4), P4(pri=2). Run P4.

P4 finishes at t=10. At t=10: P1(rem=7, pri=3), P3(pri=4). Run P1.

P1 finishes at t=17. P3 runs t=17 to 26.

Time	Running	Event
0-1	P1	Only P1; P1 runs
1-5	P2	P2 arrives (pri=1 > pri=3) → preempt P1; P2 runs
5-10	P4	P2 done; P4 has highest pri among remaining ($2 < 3 < 4$)
10-17	P1	P4 done; P1 has higher pri than P3 ($3 < 4$)
17-26	P3	P1 done; only P3 left

Process	AT	BT	Finish	TAT	WT
P1	0	8	17	17	9
P2	1	4	5	4	0
P3	2	9	26	24	15
P4	3	5	10	7	2

Gantt: | P1(0-1)|P2(1-5)|P4(5-10)|P1(10-17)|P3(17-26) |

Avg TAT = $(17+4+24+7)/4 = 52/4 = 13$ ms | Avg WT = $(9+0+15+2)/4 = 26/4 = 6.5$ ms

———— FCFS (same processes) ————

Order: P1(t=0)→P2(t=1)→P3(t=2)→P4(t=3), run in order of arrival.

Process	Start	Finish	TAT	WT
P1	0	8	8	0

Process	Start	Finish	TAT	WT
P2	8	12	11	7
P3	12	21	19	10
P4	21	26	23	18

Avg TAT = $(8+11+19+23)/4 = 15.25$ ms | Avg WT = $(0+7+10+18)/4 = 8.75$ ms

Q42. Preemptive SJF (SRTF) and Round Robin — Solved Problem. [10 Marks]

Given (TQ=3 for RR):

Process	AT	BT
P1	0	7
P2	2	4
P3	4	1
P4	5	4

———— Preemptive SJF (Shortest Remaining Time First / SRTF) ————

Time	Running	Event / Remaining BTs
0-2	P1	P1 starts (only process). Remaining: P1=7
2	—	P2 arrives (BT=4 < P1 remaining=5). Preempt P1!
2-4	P2	P2 runs. Remaining: P1=5, P2=4
4	—	P3 arrives (BT=1 < P2 remaining=2). Preempt P2!
4-5	P3	P3 runs (BT=1). P3 DONE at t=5
5	—	P4 arrives (BT=4). Compare: P2=2, P1=5, P4=4. Run P2.
5-7	P2	P2 finishes at t=7
7	—	Remaining: P1=5, P4=4. Run P4 (shorter).
7-11	P4	P4 DONE at t=11
11-16	P1	P1 DONE at t=16

Process	AT	BT	Finish	TAT	WT
P1	0	7	16	16	9
P2	2	4	7	5	1
P3	4	1	5	1	0
P4	5	4	11	6	2

Gantt: | P1(0-2)|P2(2-4)|P3(4-5)|P2(5-7)|P4(7-11)|P1(11-16) |

Avg TAT = $(16+5+1+6)/4 = 28/4 = 7$ ms | Avg WT = $(9+1+0+2)/4 = 12/4 = 3$ ms

Round Robin (TQ=3)

Time	Running	Remaining: P1 P2 P3 P4
0-3	P1	4 4 1 4
3-5	P2 (arrives at 2)	4 1 1 4 (P3 arrived at 4)
5-6	P3	4 1 0 4 → P3 DONE at 6
6-9	P4 (arrived at 5)	4 1 0 1
9-12	P1	1 1 0 1
12-13	P2 (1ms left)	1 0 0 1 → P2 DONE at 13
13-14	P4 (1ms left)	1 0 0 0 → P4 DONE at 14
14-15	P1 (1ms left)	0 0 0 0 → P1 DONE at 15

Process	Finish	TAT	WT
P1	15	15	8
P2	13	11	7
P3	6	2	1
P4	14	9	5

Avg TAT = $(15+11+2+9)/4 = 37/4 = 9.25$ ms | Avg WT = $(8+7+1+5)/4 = 21/4 = 5.25$ ms

Q43. Illustrate how TestAndSet satisfies all three critical section criteria. [10 Marks]

TestAndSet is an atomic hardware instruction that reads a memory location and sets it to TRUE in one uninterruptible operation.

TestAndSet Instruction Definition:

```
boolean TestAndSet(boolean *target) {
    boolean rv = *target; // read current value
    *target = true;       // set to true atomically
    return rv;           // return original value
}
```

Usage for Mutual Exclusion:

```
// Shared:
boolean lock = false;

// Process Pi:
do {
    while (TestAndSet(&lock)); // busy wait until lock=false
    // *** CRITICAL SECTION ***
    lock = false;             // release lock
    // *** REMAINDER SECTION ***
} while (true);
```

1. Mutual Exclusion — Satisfied:

Suppose P1 is in CS (lock=true). P2 calls TestAndSet(&lock) → reads true (lock was true) → sets lock=true again → returns true → P2 stays in while loop (busy waits). Only when P1 sets lock=false can P2 get TestAndSet to return false and enter CS. So only one process in CS at a time.

2. Progress — Satisfied:

If no process is in CS (lock=false), the first process to call TestAndSet gets rv=false (lock was false) → exits while loop → enters CS. The lock is set to true atomically, so no two processes simultaneously see lock=false. CS is always eventually entered by someone wanting it.

3. Bounded Waiting — Problem (simple version):

Simple TestAndSet does NOT guarantee bounded waiting — a process could be skipped indefinitely if another process keeps re-entering. Solution: Add a waiting array.

TestAndSet with Bounded Waiting:

```
boolean waiting[n] = {false};
boolean lock = false;

// Entry for Pi:
waiting[i] = true;
key = true;
while (waiting[i] && key)
    key = TestAndSet(&lock);
waiting[i] = false;
// *** CRITICAL SECTION ***
// Exit:
j = (i+1) % n;
while (j != i && !waiting[j]) j = (j+1)%n;
if (j == i) lock = false;
else waiting[j] = false; // hand off to next waiting process
```

With waiting array: A process enters CS within n-1 turns after it starts waiting → Bounded Waiting satisfied.

Criterion	Simple TestAndSet	TestAndSet + Waiting Array
Mutual Exclusion	✓ Guaranteed	✓ Guaranteed
Progress	✓ Guaranteed	✓ Guaranteed
Bounded Waiting	✗ Not guaranteed	✓ Guaranteed ($\leq n-1$ turns)

Q44. Illustrate the Dining Philosophers problem and derive a solution using semaphores. [10 Marks]

Problem Statement: 5 philosophers sit at a round table. Between each pair of adjacent philosophers is one chopstick (5 total). Each philosopher alternates between thinking and eating. To eat, a philosopher needs BOTH left and right chopsticks. After eating, both chopsticks are put back.

Challenge: Design a protocol that prevents deadlock and starvation.

Naive Solution (Causes Deadlock):

```
semaphore chopstick[5] = {1,1,1,1,1};
// Philosopher i:
do {
    think();
    wait(chopstick[i]);    // pick up left
```

```

wait(chopstick[(i+1)%5]); // pick up right
eat();
signal(chopstick[i]);
signal(chopstick[(i+1)%5]);
} while(true);

```

Deadlock: If all 5 philosophers simultaneously pick up their LEFT chopstick — each holds one, waiting for the right one (held by neighbor). Circular wait → Deadlock.

Solution 1: Allow at most 4 philosophers to sit simultaneously

```

semaphore chopstick[5] = {1,1,1,1,1};
semaphore room = 4; // at most 4 at table
// Philosopher i:
do {
    think();
    wait(room);
    wait(chopstick[i]);
    wait(chopstick[(i+1)%5]);
    eat();
    signal(chopstick[(i+1)%5]);
    signal(chopstick[i]);
    signal(room);
} while(true);

```

With only 4 at table: at least one philosopher can always eat (pigeonhole principle) → no deadlock.

Solution 2: Asymmetric Solution

- Odd-numbered philosophers pick up LEFT then RIGHT
- Even-numbered philosophers pick up RIGHT then LEFT
- This breaks circular wait — deadlock impossible

Solution 3: Using Monitor (state-based):

```

enum State { THINKING, HUNGRY, EATING };
State state[5];
semaphore self[5] = {0}; // one per philosopher
semaphore mutex = 1;

void pickup(int i) {
    wait(mutex);
    state[i] = HUNGRY;
    test(i); // try to eat
    signal(mutex);
    wait(self[i]); // wait if can't eat
}

void test(int i) {
    if (state[i]==HUNGRY &&
        state[(i+4)%5]!=EATING &&
        state[(i+1)%5]!=EATING) {
        state[i] = EATING;
        signal(self[i]);
    }
}

```

```

void putdown(int i) {
    wait(mutex);
    state[i] = THINKING;
    test((i+4)%5); // test left neighbor
    test((i+1)%5); // test right neighbor
    signal(mutex);
}

```

This solution ensures no deadlock and no starvation (neighbors are checked on putdown).

Q45. Explain the two ways deadlock can be detected. Illustrate recovery from deadlock. [10 Marks]

1. Detection for Single-Instance Resources — RAG Cycle Detection

Maintain a Resource Allocation Graph. Use cycle detection algorithm (DFS):

69. Build adjacency list for RAG
70. Run DFS from every unvisited node
71. Track visited nodes and recursion stack
72. If a node is visited again while on recursion stack → cycle → deadlock

Example: $P1 \rightarrow R1 \rightarrow P2 \rightarrow R2 \rightarrow P1 \rightarrow \text{cycle} \rightarrow \text{deadlock among } P1, P2$

2. Detection for Multiple-Instance Resources — Matrix Algorithm

Uses: Available[], Allocation[[[]], Request[[[]] matrices.

73. Initialize Work = Available. Finish[i] = false for all i.
74. Find P_i such that Finish[i]=false AND Request[i] ≤ Work. If found: Work = Work + Allocation[i]; Finish[i] = true. Repeat.
75. If Finish[i] = false for some i → P_i is deadlocked.

Example with 5 processes, 3 resources (A=7,B=2,C=6):

P	Allocation (A,B,C)	Request (A,B,C)
P0	0,1,0	0,0,0
P1	2,0,0	2,0,2
P2	3,0,3	0,0,0
P3	2,1,1	1,0,0
P4	0,0,2	0,0,2

Work = Available = (0,0,0). Finish = [F,F,F,F,F]

P0: Request(0,0,0) ≤ Work(0,0,0) → YES → Work=(0,1,0), Finish[0]=T

P2: Request(0,0,0) ≤ Work(0,1,0) → YES → Work=(3,1,3), Finish[2]=T

P3: Request(1,0,0) ≤ Work(3,1,3) → YES → Work=(5,2,4), Finish[3]=T

P1: Request(2,0,2) ≤ Work(5,2,4) → YES → Work=(7,2,4), Finish[1]=T

P4: Request(0,0,2) ≤ Work(7,2,4) → YES → Finish[4]=T

All Finish = T → NO DEADLOCK in this example.

Recovery from Deadlock:

Method 1: Process Termination

- a) Abort all deadlocked processes: Expensive — all work lost, but guaranteed to break deadlock immediately.
- b) Abort one process at a time: Run detection after each abort. Use victim selection: abort process

with minimum cost (least progress, lowest priority, fewest resources held).

Method 2: Resource Preemption

- Select victim: choose process with minimum cost to preempt resources from.
- Rollback: Return victim process to a safe checkpoint state (requires periodic checkpointing).
- Starvation prevention: Ensure same process is not always chosen as victim (track number of times preempted).

Q46. Banker's Algorithm — Solved Problem. [10 Marks]

Given: 5 processes (P0-P4), 3 resource types A, B, C. Total resources: A=10, B=5, C=7.

Process	Max A,B,C	Allocated A,B,C	Need = Max-Alloc
P0	7,5,3	0,1,0	7,4,3
P1	3,2,2	2,0,0	1,2,2
P2	9,0,2	3,0,2	6,0,0
P3	2,2,2	2,1,1	0,1,1
P4	4,3,3	0,0,2	4,3,1

Total Allocated = (7,2,5). Available = Total - Allocated = (10,5,7)-(7,2,5) = (3,3,2)

Step 1: Run Safety Algorithm

Work = (3,3,2). Finish = [F,F,F,F,F]

Step	Process	Need ≤ Work?	Work after finish	Finish
1	P1	(1,2,2) ≤ (3,3,2)? YES	(3,3,2)+(2,0,0)=(5,3,2)	P1=T
2	P3	(0,1,1) ≤ (5,3,2)? YES	(5,3,2)+(2,1,1)=(7,4,3)	P3=T
3	P4	(4,3,1) ≤ (7,4,3)? YES	(7,4,3)+(0,0,2)=(7,4,5)	P4=T
4	P0	(7,4,3) ≤ (7,4,5)? YES	(7,4,5)+(0,1,0)=(7,5,5)	P0=T
5	P2	(6,0,0) ≤ (7,5,5)? YES	P2 done	P2=T

Safe Sequence: P1 → P3 → P4 → P0 → P2 ✓ SAFE STATE

Step 2: Resource Request — P1 requests (1,0,2)

76. Check: Request(1,0,2) ≤ Need[P1](1,2,2)? YES ✓

77. Check: Request(1,0,2) ≤ Available(3,3,2)? YES ✓

78. Tentatively allocate: Available=(3,3,2)-(1,0,2)=(2,3,0). Allocated[P1]=(2,0,0)+(1,0,2)=(3,0,2).
Need[P1]=(1,2,2)-(1,0,2)=(0,2,0)

79. Run safety algorithm with new state → safe sequence still exists

Result: REQUEST GRANTED. System remains in safe state.

Q47. Distinguish: Logical/Physical | Multiple/Variable partition | First/Best/Worst Fit. [10 Marks]

A. Logical vs Physical Address Space

Feature	Logical Address	Physical Address
---------	-----------------	------------------

Feature	Logical Address	Physical Address
Definition	CPU-generated address during instruction execution	Actual location in RAM
Seen by	User process / programmer	Memory hardware
Size	Can be larger than physical RAM (virtual memory)	Limited by actual RAM installed
Example	Process generates address 500	MMU translates to 14500 (base=14000+500)

B. Multiple Contiguous (Fixed) Partition vs Variable Partition Allocation

Feature	Fixed/Multiple Partition	Variable Partition
Partition size	Fixed at boot time	Varies — exactly as needed
Internal fragmentation	Yes — process smaller than partition wastes space	No
External fragmentation	No — fixed boundaries	Yes — holes between allocations
Flexibility	Low — fixed sizes	High
Implementation	Simple	More complex (tracking holes)
Example	Partition of 50KB holds a 30KB process (20KB wasted)	30KB process gets exactly 30KB partition

C. First Fit, Best Fit, Worst Fit (Contiguous Variable Partition Allocation)

Memory holes available: 100KB, 500KB, 200KB, 300KB, 600KB. Process needs 212KB.

Algorithm	Strategy	Hole chosen (212KB)	Remaining hole	Speed
First Fit	First hole $\geq$ request	500KB hole	288KB	Fastest
Best Fit	Smallest hole $\geq$ request	300KB hole	88KB	Slower (search all)
Worst Fit	Largest hole $\geq$ request	600KB hole	388KB	Slowest (search all)

- First Fit: Fastest. May leave large holes for future large requests. Generally good.
- Best Fit: Minimizes wasted space per allocation but may create many tiny unusable holes.
- Worst Fit: Leaves largest remaining holes — good when future requests are expected to be large.

Q48. Describe paging. Explain page table structure and address translation. [10 Marks]

(Core content same as B14 — extended with address translation example.)

Paging divides physical memory into fixed-size frames and logical memory into same-size pages.

Page size is power of 2 — typically 4KB (2^{12}) to 4MB.

Address Translation Example:

Page size = 4KB = 2^{12} . 32-bit logical address space $\rightarrow$ 20-bit page number, 12-bit offset.

Logical address = 0x0000_3A5F

- Page number $p = 0x3 = 3$
- Page offset $d = 0xA5F = 2655$
- Page table: page 3 $\rightarrow$ frame 8
- Physical address = $8 \times 4096 + 2655 = 32768 + 2655 = 35423 = 0x0000_8A5F$

Page Table Structure:

- Each entry (PTE) contains: Frame number, Valid/Invalid bit, Protection bits (R/W/X), Reference bit (for LRU), Dirty bit (modified)

Page No	Frame No	Valid	Dirty	Protection
0	5	1	0	R
1	3	1	1	R/W
2	—	0	—	—
3	8	1	0	R/W

Hierarchical Page Table (2-Level):

- For 32-bit: 10-bit outer index, 10-bit inner index, 12-bit offset
- Outer page table (1024 entries) $\rightarrow$ Inner page table (1024 entries each) $\rightarrow$ Frame
- Only inner tables for used address space regions need to be in memory — saves space

Inverted Page Table:

- One global table: one entry per physical frame
- Entry: {PID, page_number} $\rightarrow$ frame index is implicit
- Size: proportional to RAM size, not per-process — much smaller
- Disadvantage: lookup requires searching whole table $\rightarrow$ use hash table to speed up

Q49. Problem with simple paging. How TLB solves it. Hardware diagram and example. [10 Marks]

Problem with Simple Paging

In simple paging, page table stored in main memory. Every memory access requires TWO memory accesses:

80. First access: Read page table entry from RAM to get frame number

81. Second access: Read/write the actual data at the physical address

This doubles the effective memory access time — unacceptable performance overhead.

Solution: Translation Lookaside Buffer (TLB)

TLB is a small, extremely fast hardware cache (associative memory) inside the MMU that stores recently used page-to-frame mappings.

TLB Entry: {Page Number $\rightarrow$ Frame Number} + valid/protection bits

Address Translation with TLB:

82. CPU generates logical address $\rightarrow$ extract page number p and offset d

83. Search TLB for page number p (parallel associative lookup — very fast $\sim 1\text{-}5\text{ns}$)

84. TLB HIT: Frame number found $\rightarrow$ Physical address = frame $\times$ page_size + offset $\rightarrow$ ONE extra memory access (just the data).

85. TLB MISS: Frame not in TLB $\rightarrow$ access page table in RAM to get frame $\rightarrow$ update TLB $\rightarrow$ Physical address. TWO memory accesses.

Effective Access Time Calculation:

Let: TLB access time = 20ns, Memory access time = 100ns, TLB hit rate = 95%

$EAT = hit_rate \times (TLB + mem) + (1-hit_rate) \times (TLB + 2 \times mem)$

$EAT = 0.95 \times (20+100) + 0.05 \times (20+200)$

$EAT = 0.95 \times 120 + 0.05 \times 220 = 114 + 11 = 125ns$

Without TLB: $2 \times 100 = 200ns$. With TLB: 125ns. Speed improvement = 60%!

TLB Management:

- ASID (Address Space ID): TLB entries tagged with process ID — allows multiple processes' entries in TLB simultaneously (avoids full TLB flush on every context switch)
- TLB Flush: On context switch (without ASID), entire TLB must be invalidated — costly
- TLB Reach: Total memory accessible via TLB = $TLB_size \times page_size$ (e.g., 64 entries $\times$ 4KB = 256KB)

Scenario	Memory Accesses	Time (100ns memory)
No TLB	2 per access	200ns
TLB hit	1 per access	~120ns (TLB+data)
TLB miss	2 per access + TLB update	~220ns
EAT (95% hit rate)	—	~125ns

Q50. Differentiate contiguous and non-contiguous memory allocation. Advantages and disadvantages. [10 Marks]

Feature	Contiguous Allocation	Non-Contiguous Allocation
Memory layout	Process in consecutive memory block	Process pages/segments scattered in memory
Address translation	Simple — base+offset	Requires page table or segment table
External fragmentation	Yes — holes form between allocations	No (paging) / Minimal (segmentation)
Internal fragmentation	Yes (fixed partition) / No (variable)	Small (last page)
Sharing	Difficult — entire block	Easy — share individual pages/segments
Virtual memory	Not supported easily	Foundation of virtual memory
Protection	Coarse-grained (base+limit)	Fine-grained (per-page protection bits)
Example	Fixed partition, variable partition	Paging, segmentation, seg+paging

Contiguous Allocation Details:

1. Fixed Partitioning:

- Advantage: Simple — OS just checks partition status table. No external fragmentation.
- Disadvantage: Internal fragmentation (process smaller than partition). Fixed number of processes.

2. Variable Partitioning:

- Advantage: No internal fragmentation — exact size allocated.

- Disadvantage: External fragmentation grows over time — compaction needed (costly).

Non-Contiguous Allocation Details:

1. Paging:

- Advantage: No external fragmentation. Simple allocation (any frame = any page). Enables virtual memory.
- Disadvantage: Internal fragmentation (last page), TLB overhead, large page tables.

2. Segmentation:

- Advantage: Matches logical program structure. Easy sharing and protection per segment.
- Disadvantage: External fragmentation (variable segment sizes).

3. Segmentation + Paging (Modern OS approach):

- Best of both — logical segments (user view) + paged (no external fragmentation)
- Used in Intel x86 protected mode